

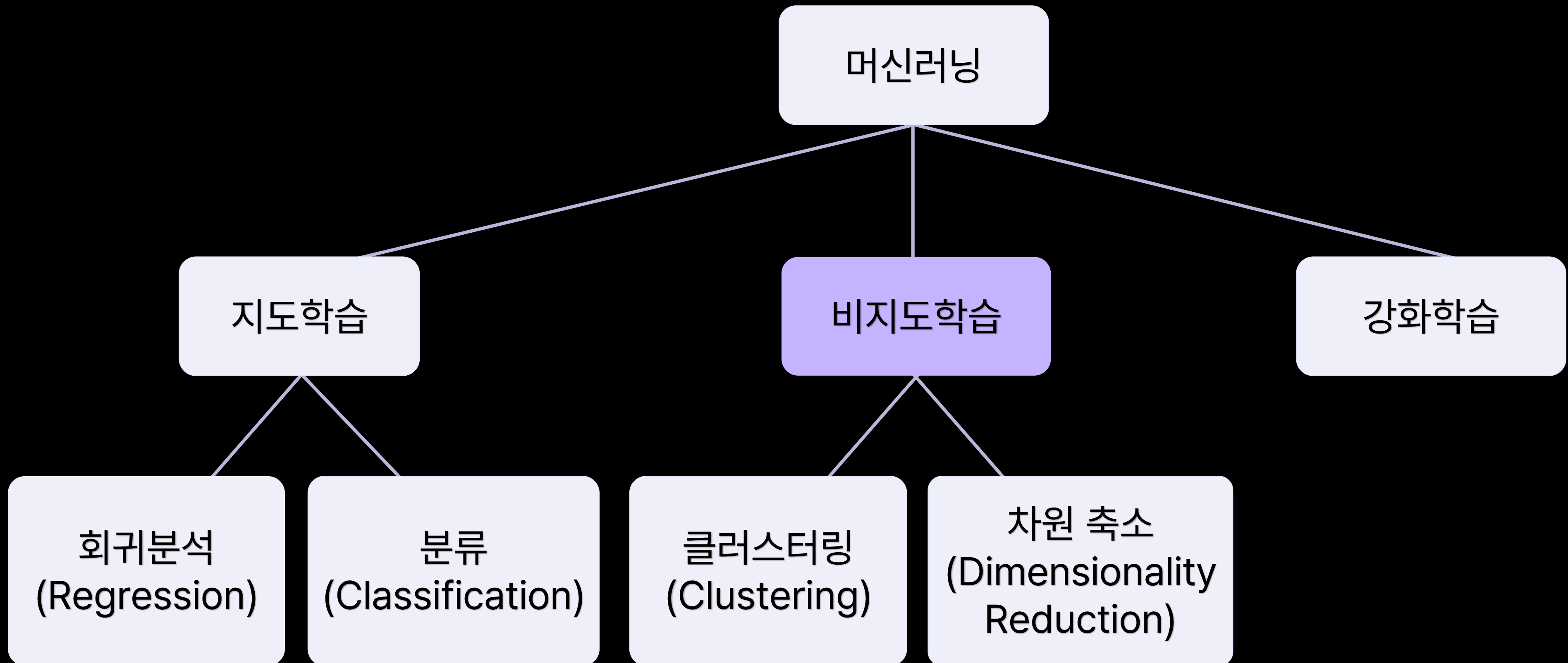
머신러닝 II

03 비지도학습

목차 03 비지도학습

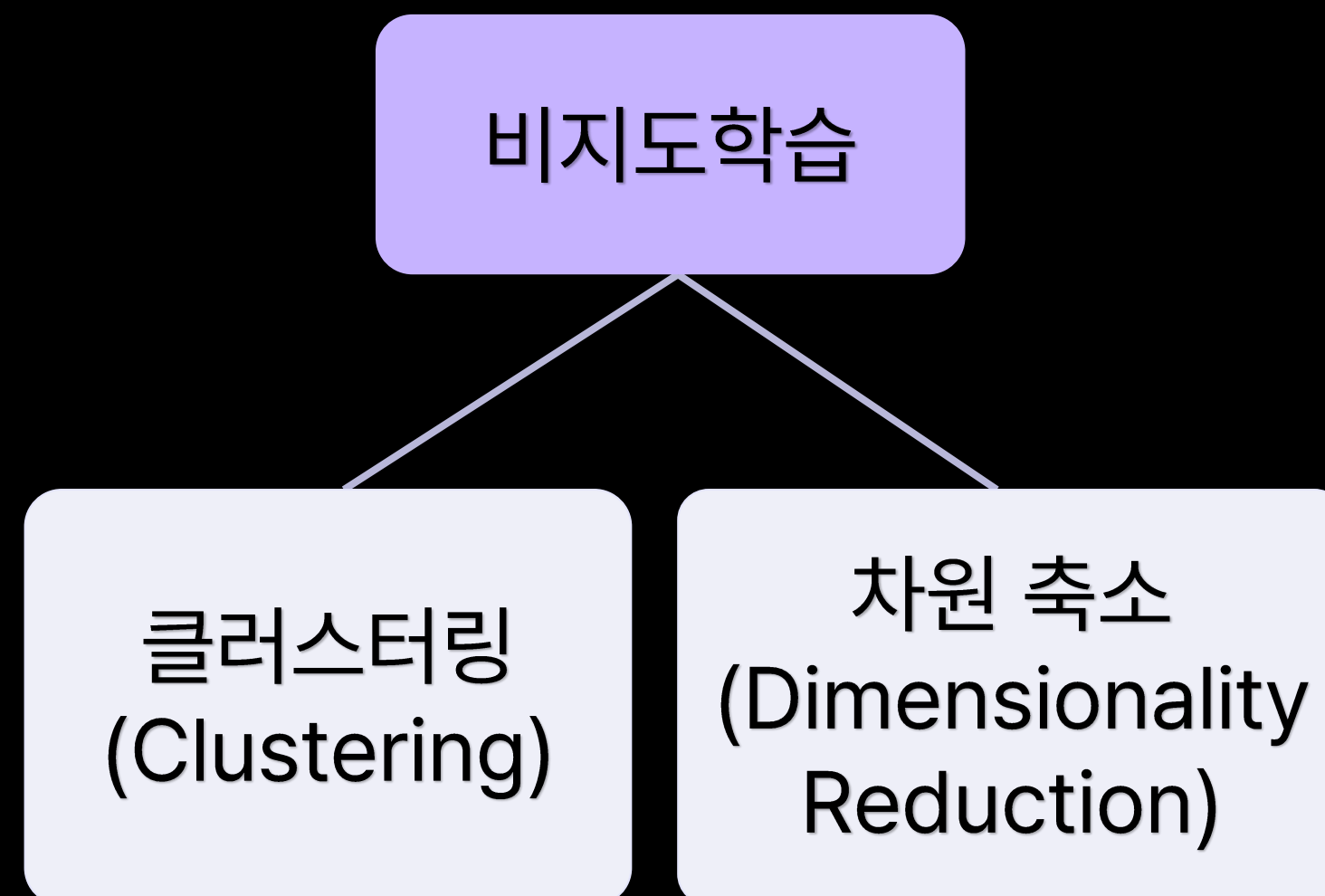
1. 비지도학습과 클러스터링
2. K-means Clustering
3. Gaussian Mixture Model(GMM)과 타당성 지표
4. 차원 축소와 주성분 분석
5. t-SNE

1. 비지도학습과 클러스터링

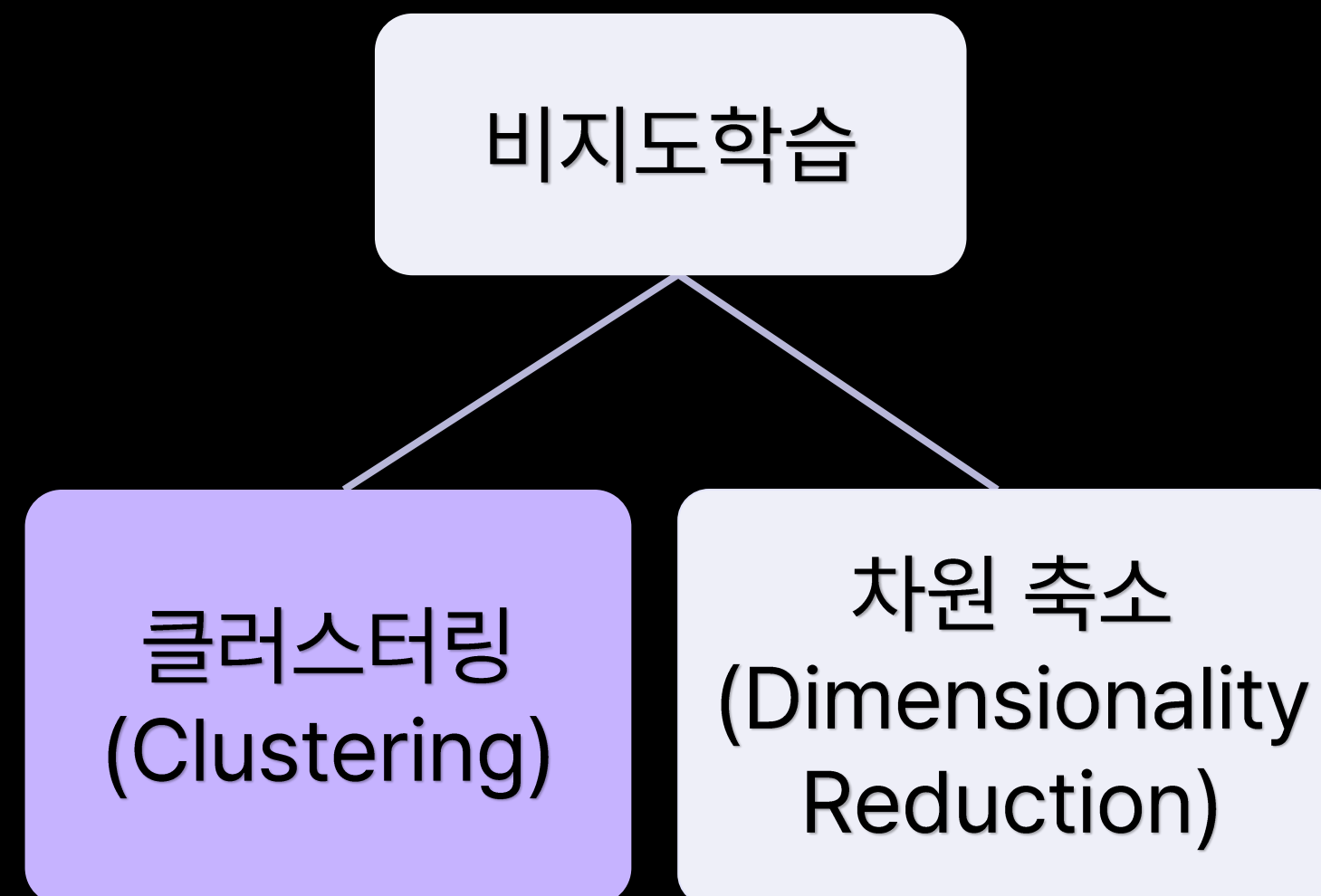


컴퓨터에 **정답이 없는 데이터**를 학습시키는 방법

- 원하는 출력 없이 입력 데이터를 사용

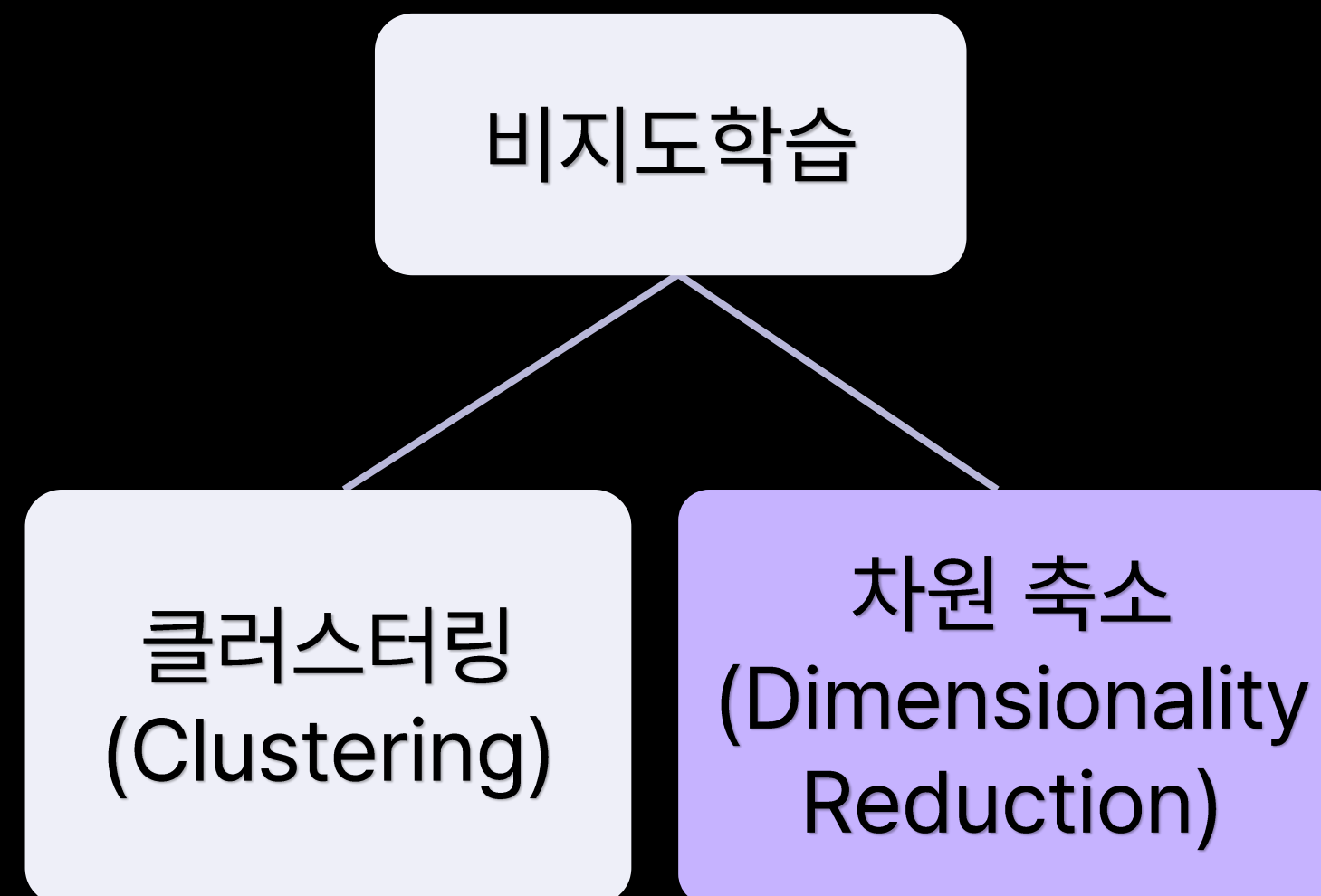


각 개체의 그룹 정보(정답) 없이 유사한 특성을 가진 개체끼리 군집화





고차원 데이터의 차원을 축소하여 데이터를 더욱 잘 설명할 수 있도록 함



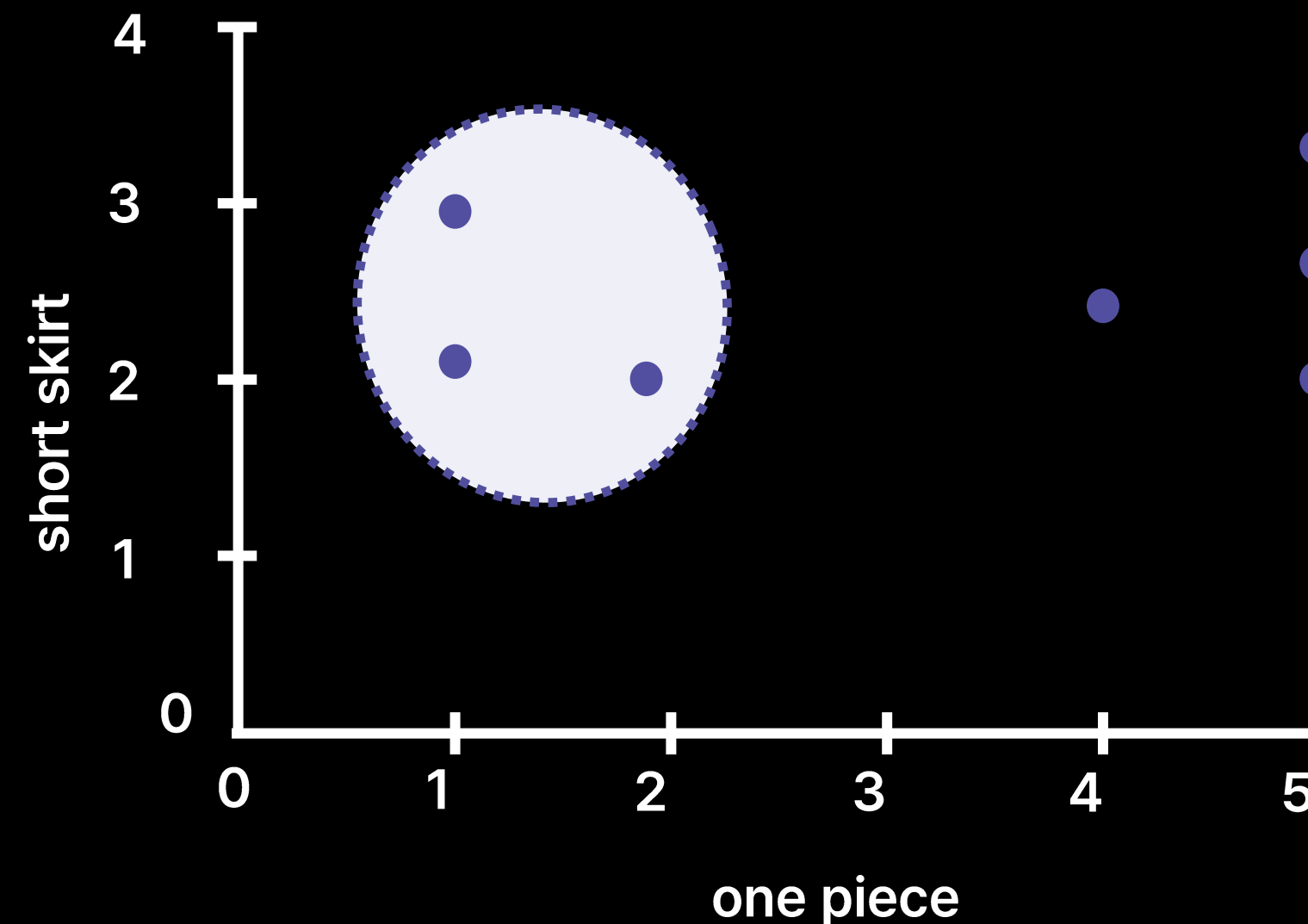
- 인터넷 쇼핑몰 마케터라고 가정
- 고객별 구매 상품 개수 데이터를 활용하여 유사한 고객 집단으로 세분화 하고자 한다면?

고객별 구매 상품 개수 데이터

index	one piece	short skirt
고객1	2	0
고객2	1	1



- 문제 정의 : 유사한 특성을 지닌 고객을 동일한 그룹으로 묶으면 어떨까?
- 데이터 : 고객별 구매 상품 개수 데이터
- 목표 : 유사한 특성을 지닌 고객 그룹화하기
- 해결 방안 : **클러스터링 알고리즘**



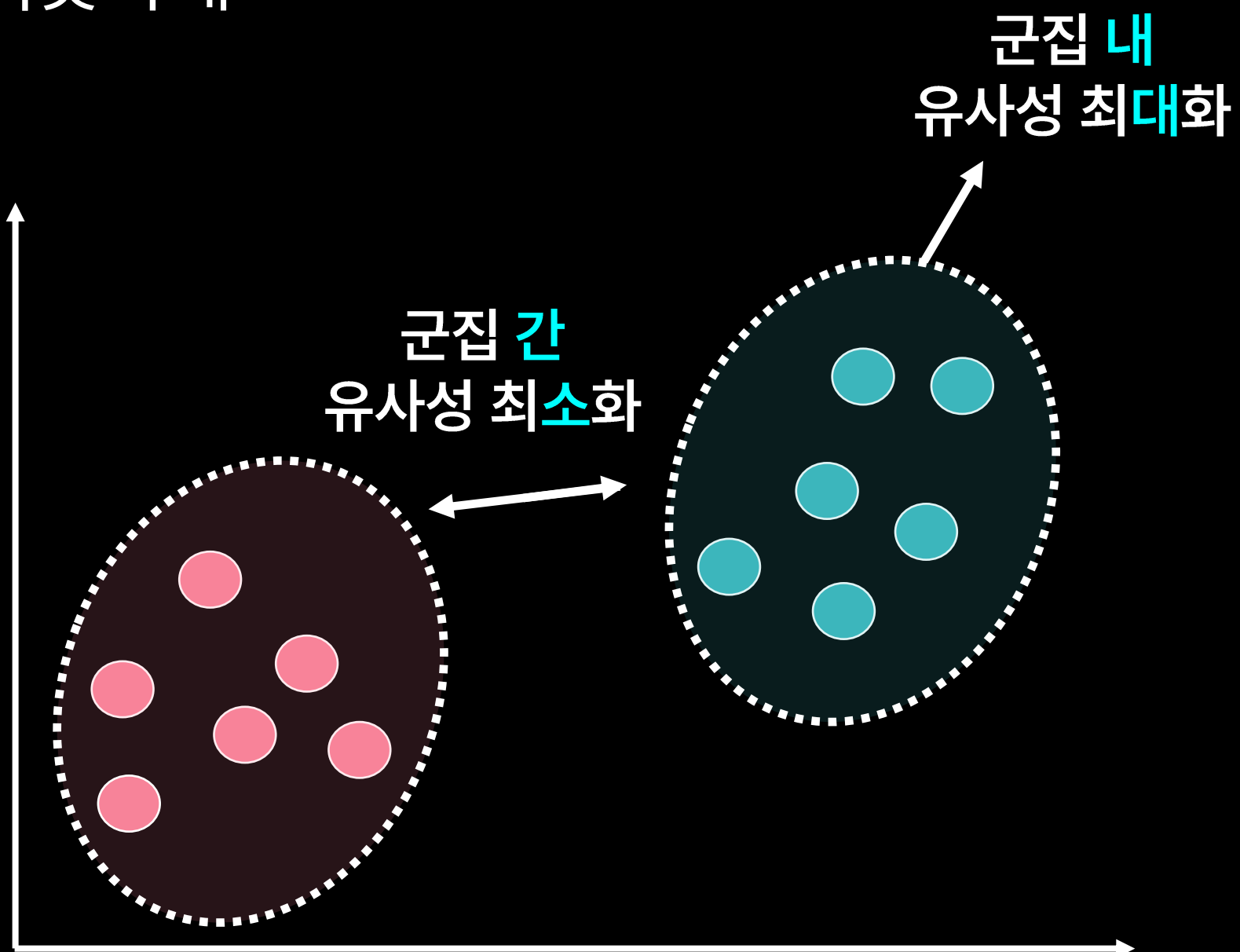


1. 군집 간 유사성 최소화

- 다른 군집 간 데이터 간에는 서로 비슷하지 않게

2. 군집 내 유사성 최대화

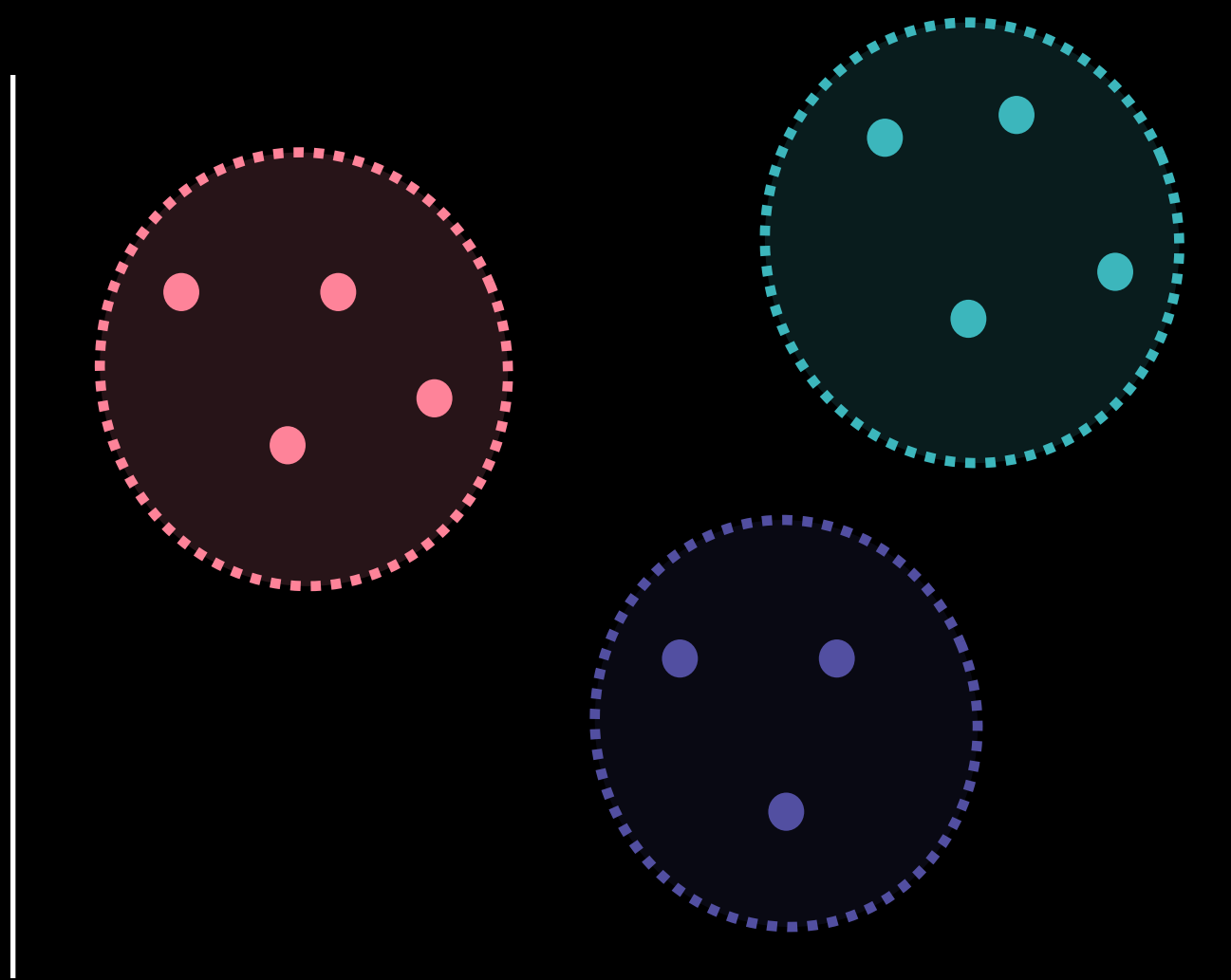
- 동일 군집 내 데이터 간에는 서로 비슷하게





각 개체의 그룹 정보(정답) 없이 유사한 특성을 가진 개체끼리 군집화

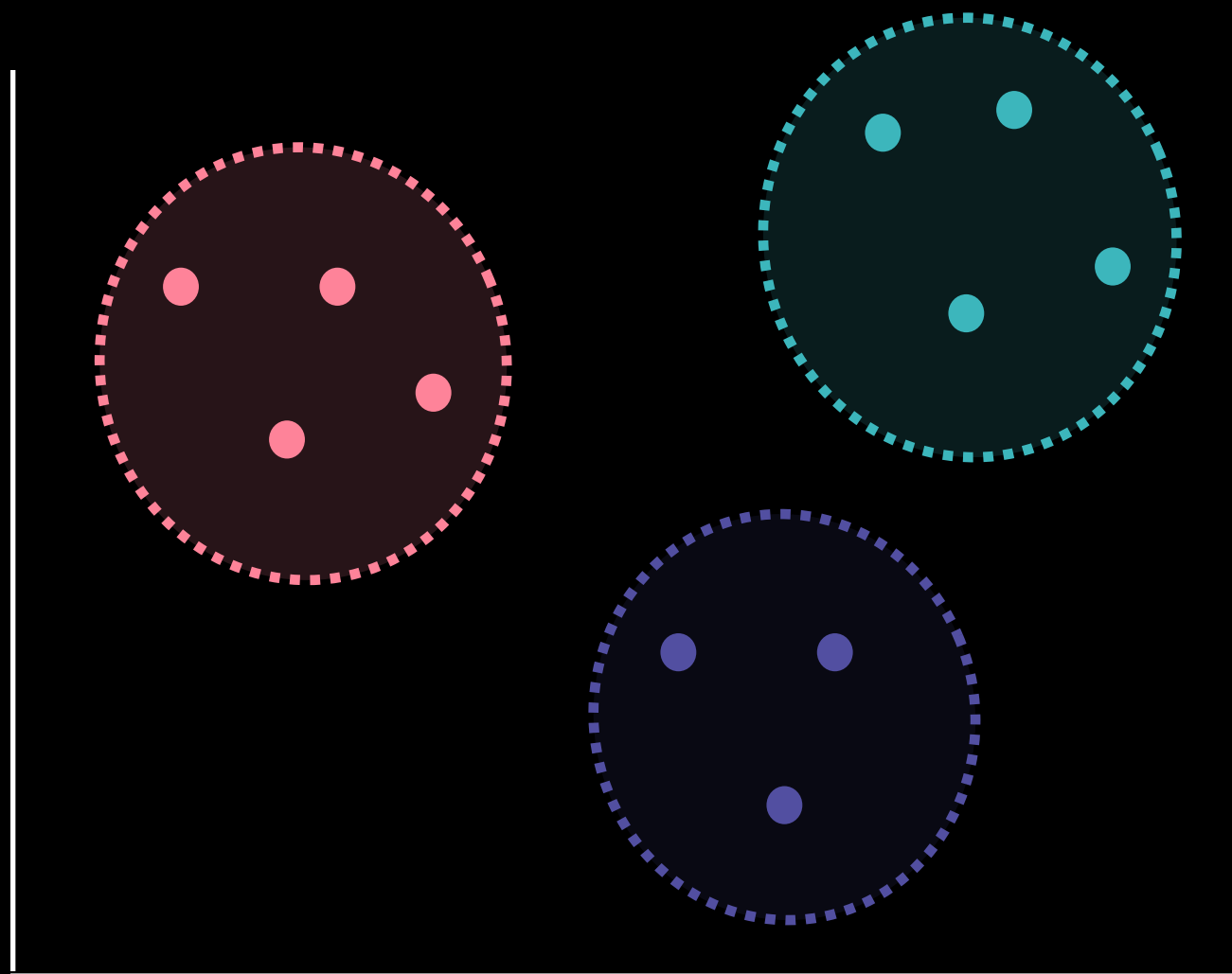
- Hard Clustering
- Soft Clustering





특정 개체가 집단에 포함되는지 여부
클러스터에 속한다(1), 속하지 않는다(0)으로 표현

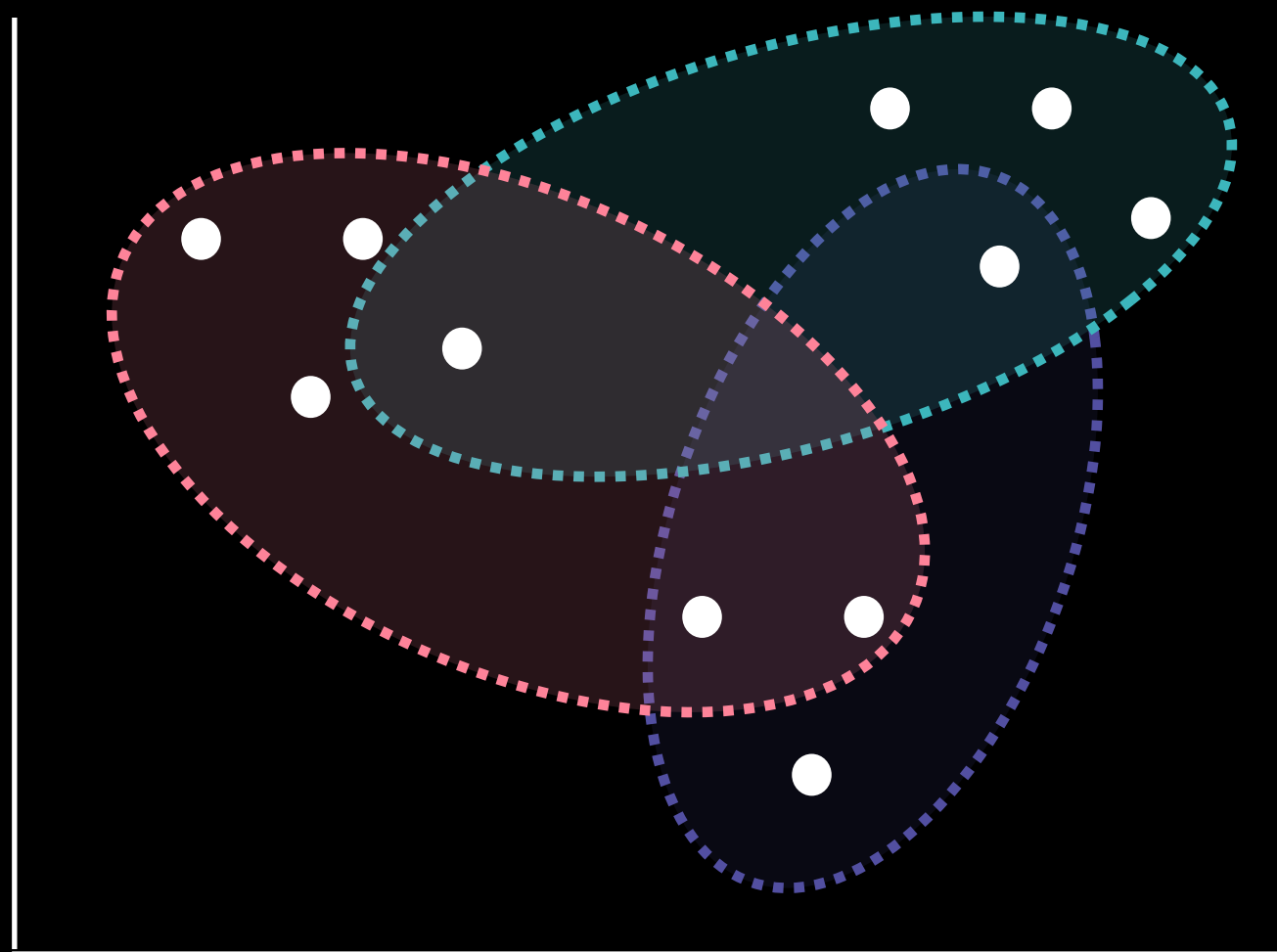
- K-means Clustering 알고리즘





특정 개체가 집단에 얼마나 포함되는지 정도 클러스터에 속하는 정도로 표현

- Gaussian Mixture Model 알고리즘



2. K-means Clustering



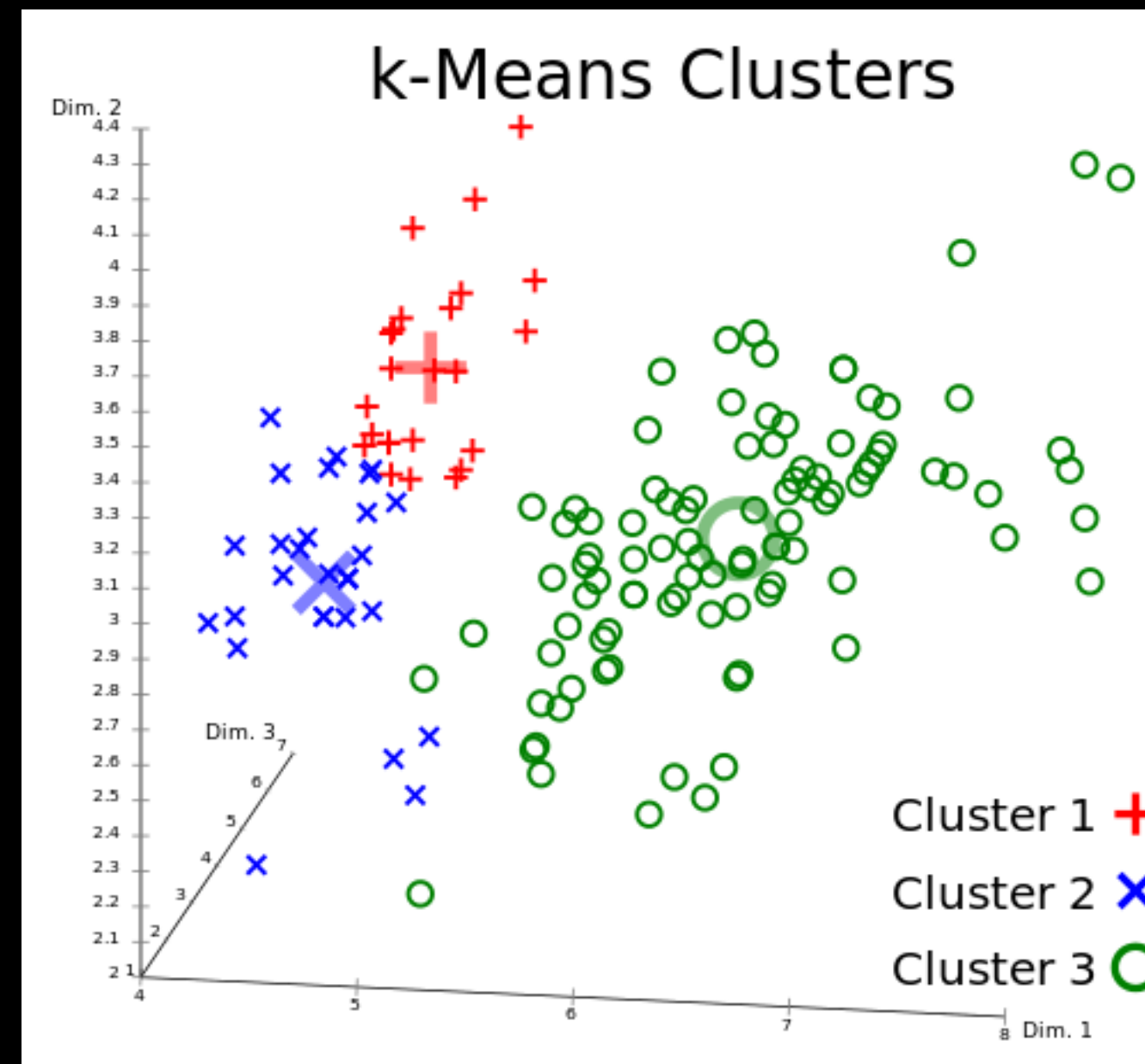
- 문제 정의 : 100만 명 이상인 고객의 구매 상품 데이터를 활용하여 고객을 군집화 하고자 한다면?
(즉, 대용량의 데이터)
- 해결 방안 : **K-means Clustering 알고리즘**





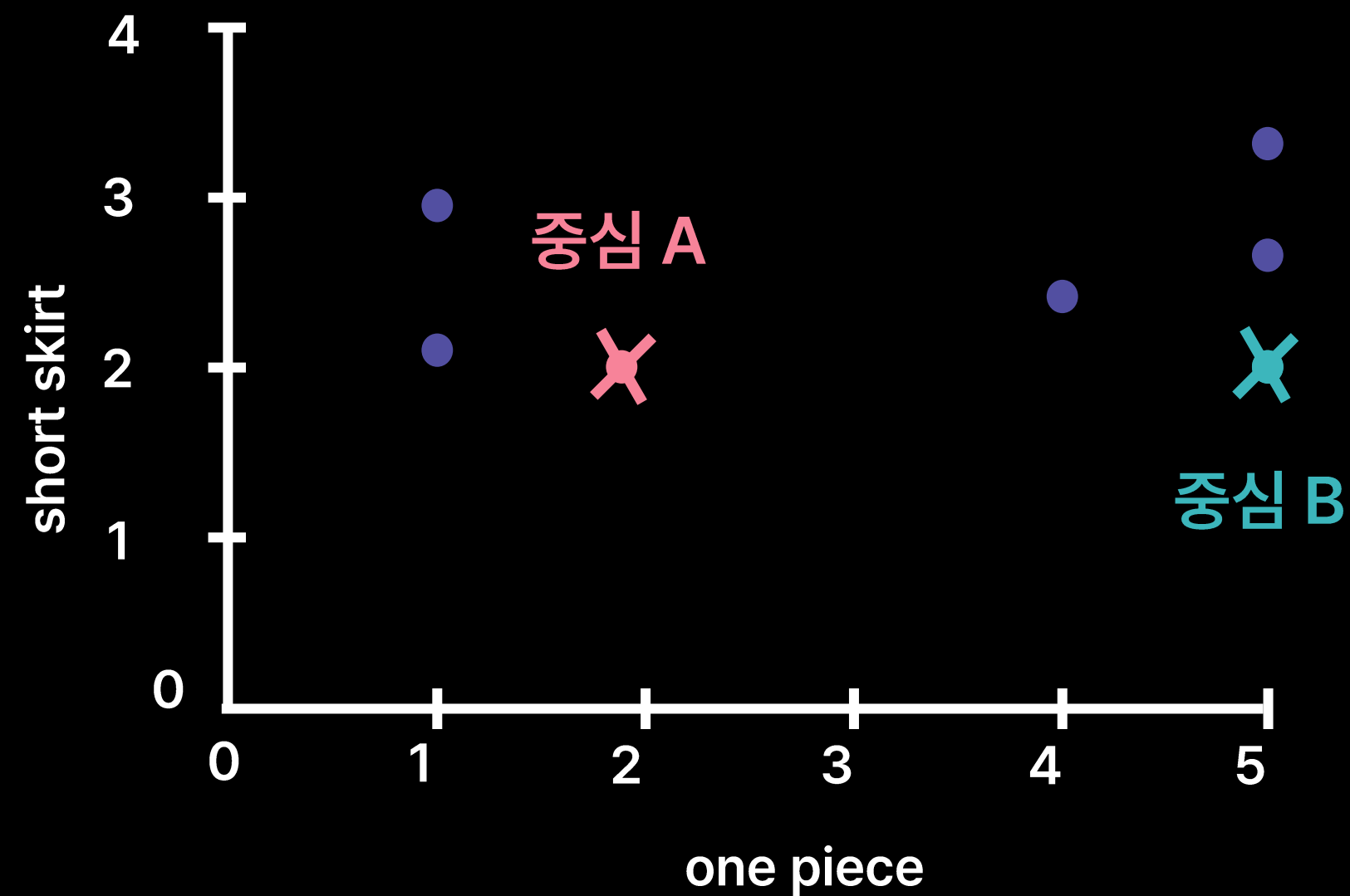
제공된 데이터를 K개로 군집화하는 알고리즘

- 군집화할 개수 K는 직접 설정해야 하는 하이퍼파라미터





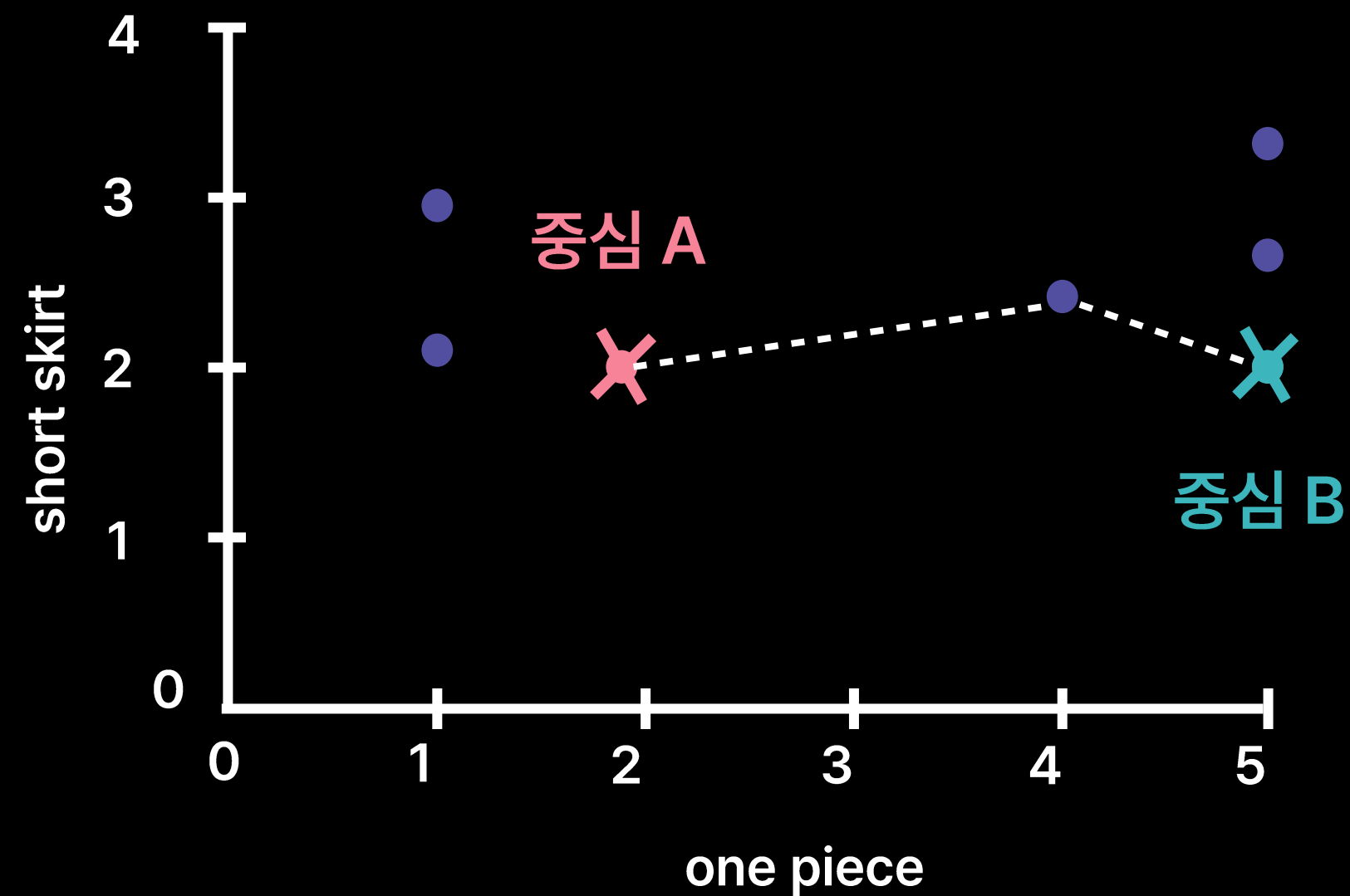
1. 데이터세트 중 **K개를 랜덤하게 뽑아** 해당 데이터를 중심으로 설정
 - 초기화 단계에서 진행





2. 모든 데이터에 대해서 아래 과정을 반복

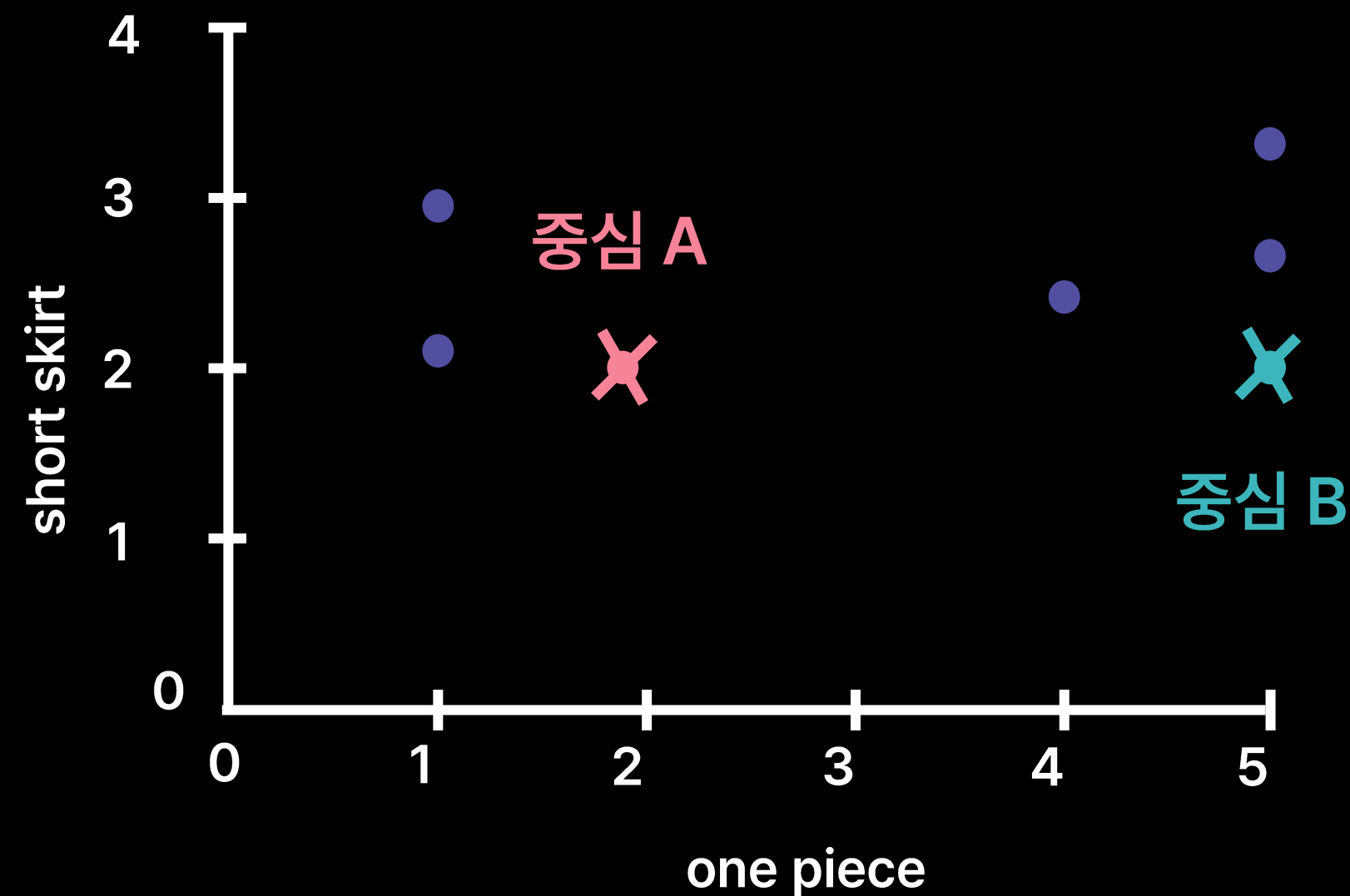
- 각 클러스터의 중심과 자신(해당 데이터)을 비교하고, 가장 가까운 클러스터를 저장





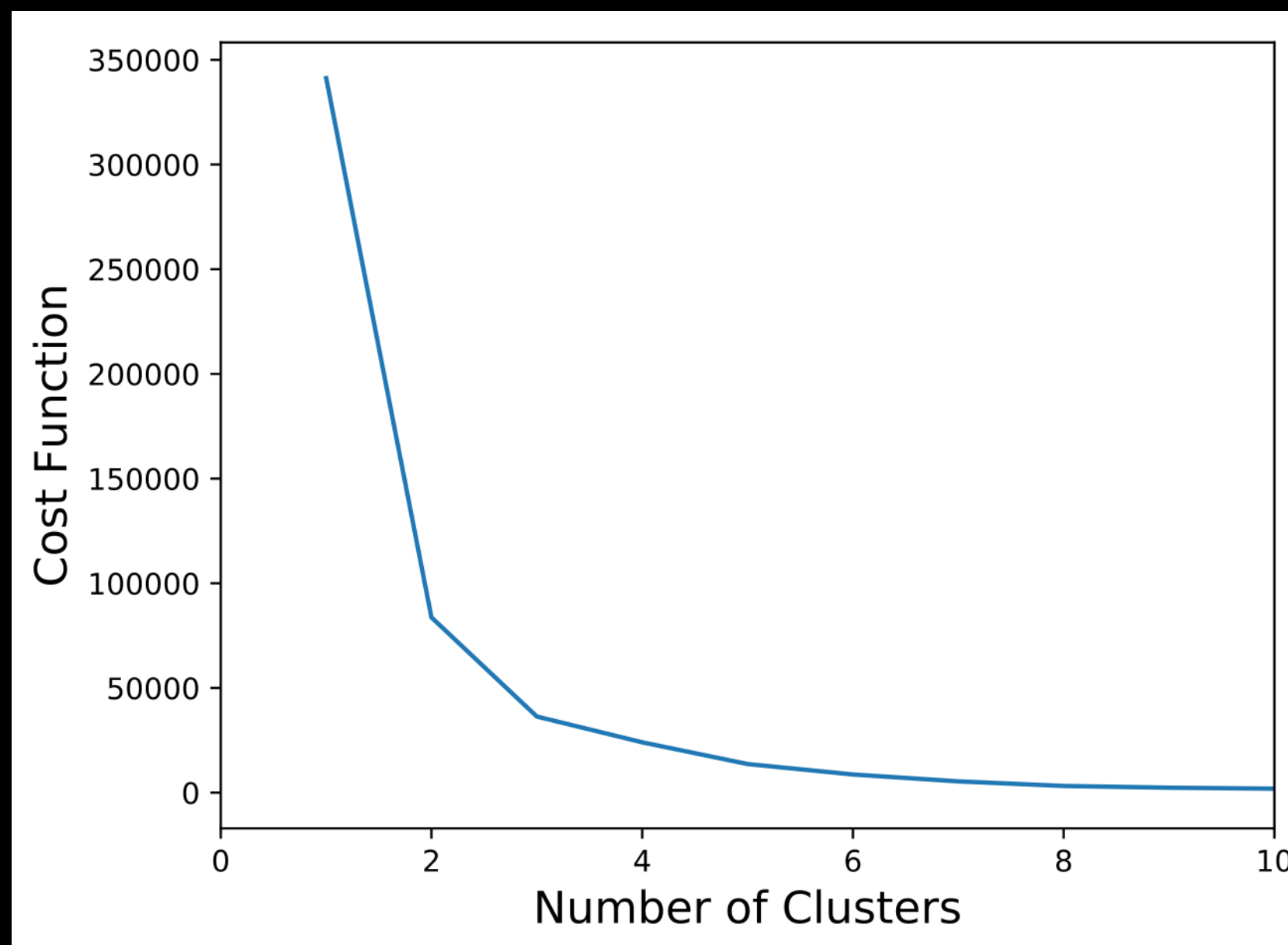
3. 모든 클러스터에 대해서 아래 과정을 반복

- 자신(해당 클러스터)에 할당된 데이터들의 중심을 계산
- 계산된 중심을 새로운 중심으로 설정, 중심의 변화가 없을 때까지 반복





- Elbow Method
 - 다양한 K값을 시도해보고, 비용 함수 그래프가 꺾이는 부분을 확인
 - 클러스터 수를 증가시켜도 효과가 별 효과가 없는 지점의 K





- **랜덤 초기값 설정**으로 인해 데이터의 분포가 독특한 경우 원하는 결과 나오지 않을 가능성 존재
 - 데이터의 분포가 원형이 아닐 때(타원형 등)
- 모델 학습 전 **변수들의 단위를 스케일링**해주는 것이 좋음
- 시간 복잡도가 가벼워 **많은 계산량이 필요한 대용량 데이터에 적합**
- 실제 문제에 적용 시, 여러 번 클러스터링을 수행해 **가장 빈번히 등장하는 군집에 할당**
- **이상치에 민감**하게 반응



```
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(init='random', n_clusters=3, random_state=100)
```

 모델 생성

```
kmeans.fit(X)
```

 모델 학습

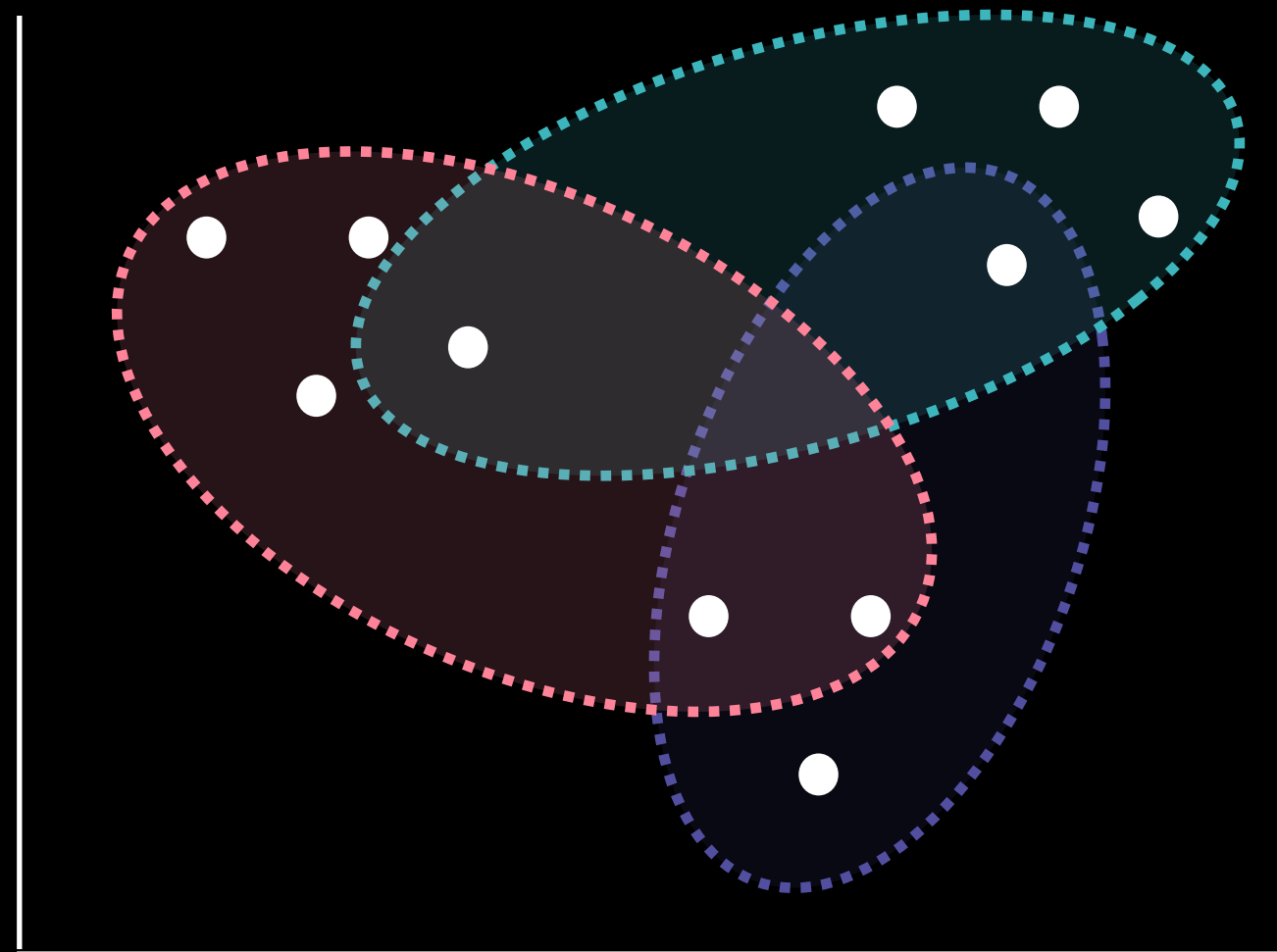
```
kmeans.labels_
```

- init = 중심점 초기화 방법 설정 { 'random', 'k-means++' }
- **n_clusters = 군집 개수 설정**
- random_state = 분할 과정에서의 무작위성 제어
- **labels_ : 각 데이터가 속한 군집의 중심점 label 반환**

3. Gaussian Mixture Model(GMM)과 타당성 지표

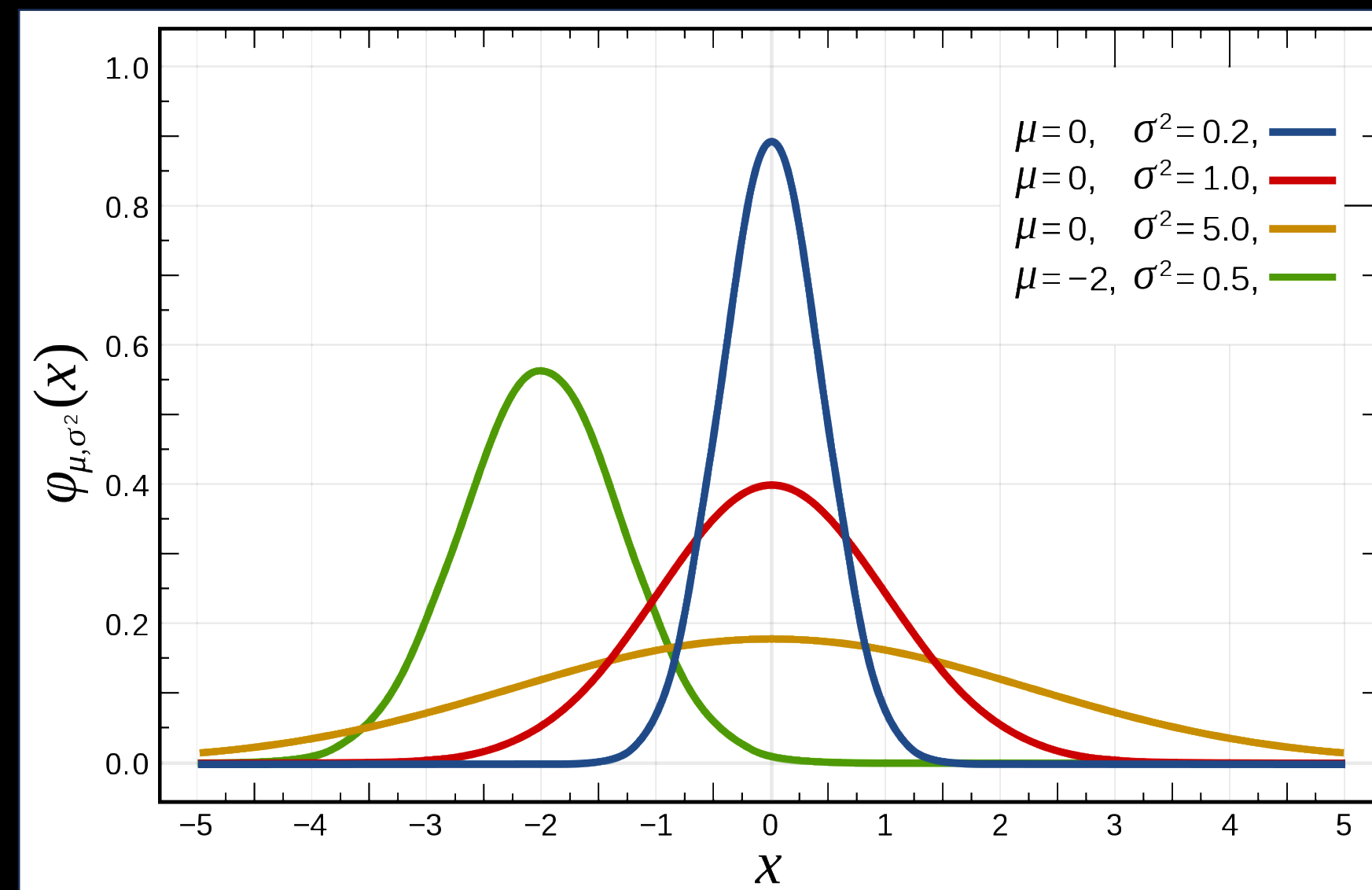


- 문제 정의 : 클러스터에 속하는 정도를 표현하는 클러스터링 알고리즘은 있을까?
- 해결 방안 : 확률을 통해 데이터 유사성을 측정하는 **Gaussian Mixture Model 알고리즘**



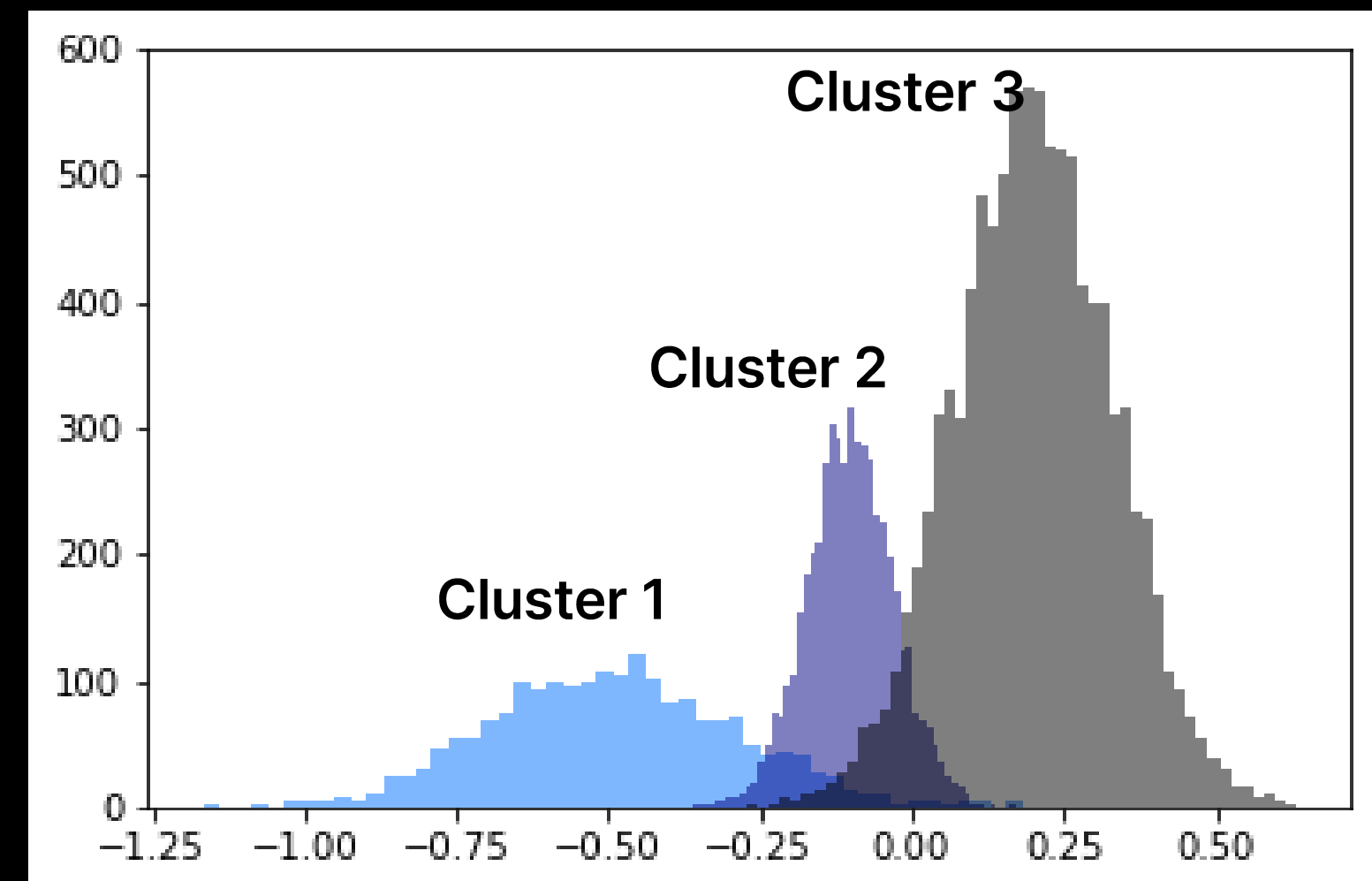
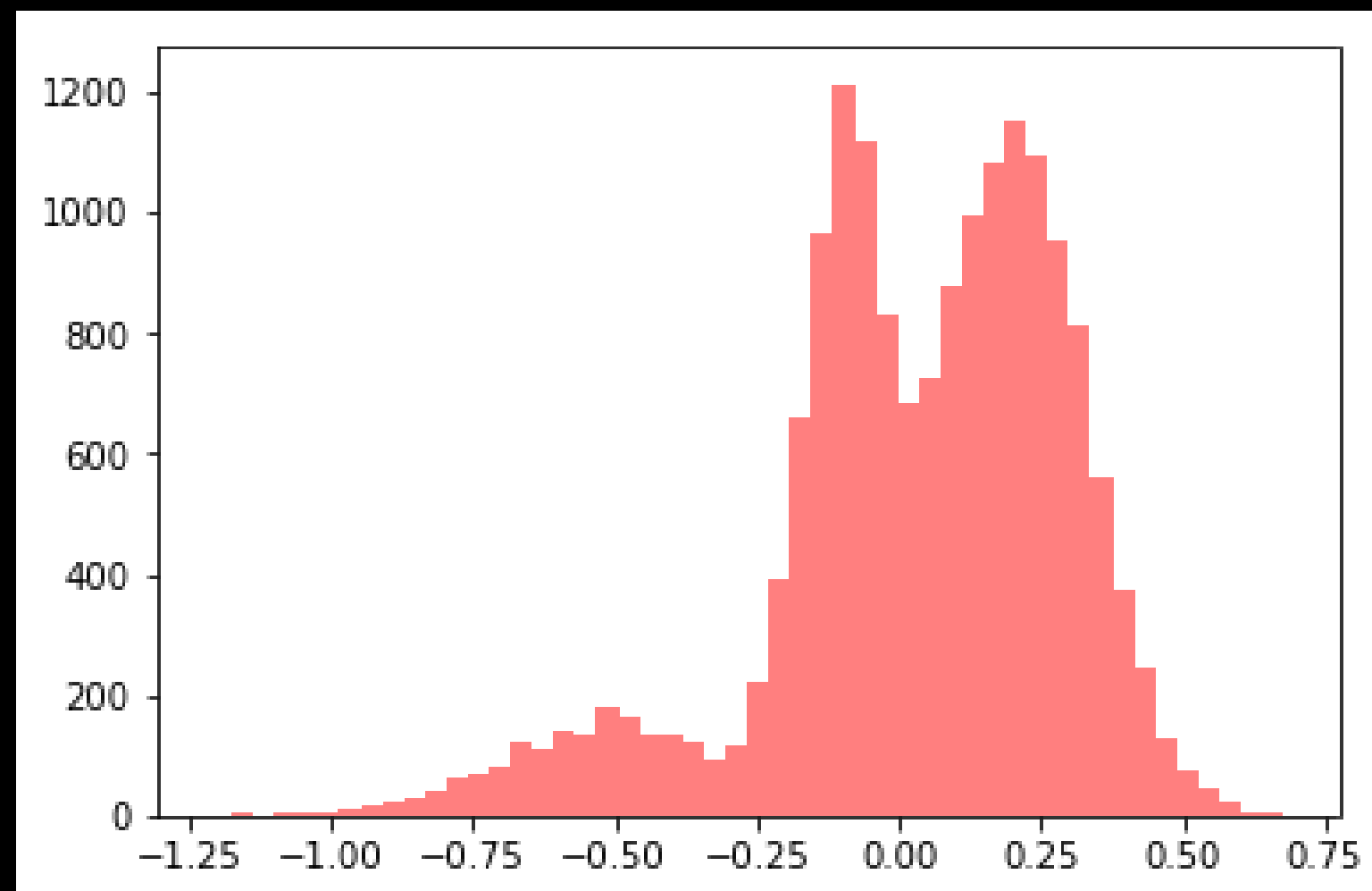


전체 데이터의 확률분포가 **여러 개의 정규 분포의 조합**으로 이루어져있다 가정하고,
각 분포에 속할 확률이 높은 데이터끼리 클러스터링하는 방법





1. 학습 데이터의 분포와 유사한 K개의 정규 분포를 추출
2. 개별 데이터가 **어떤 정규 분포에 속하는지** 결정
 - K개의 정규 분포는 K개의 클러스터에 해당



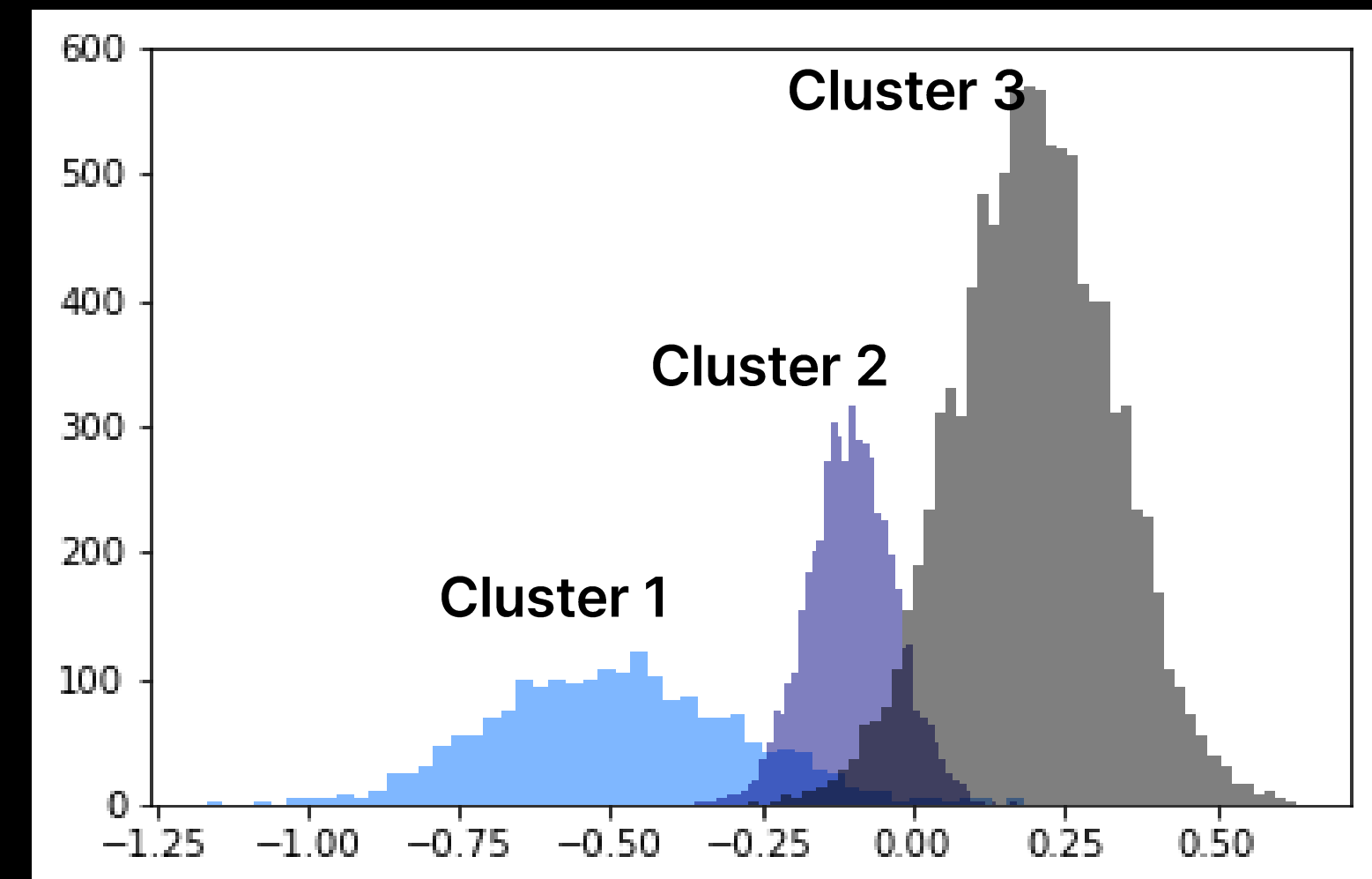
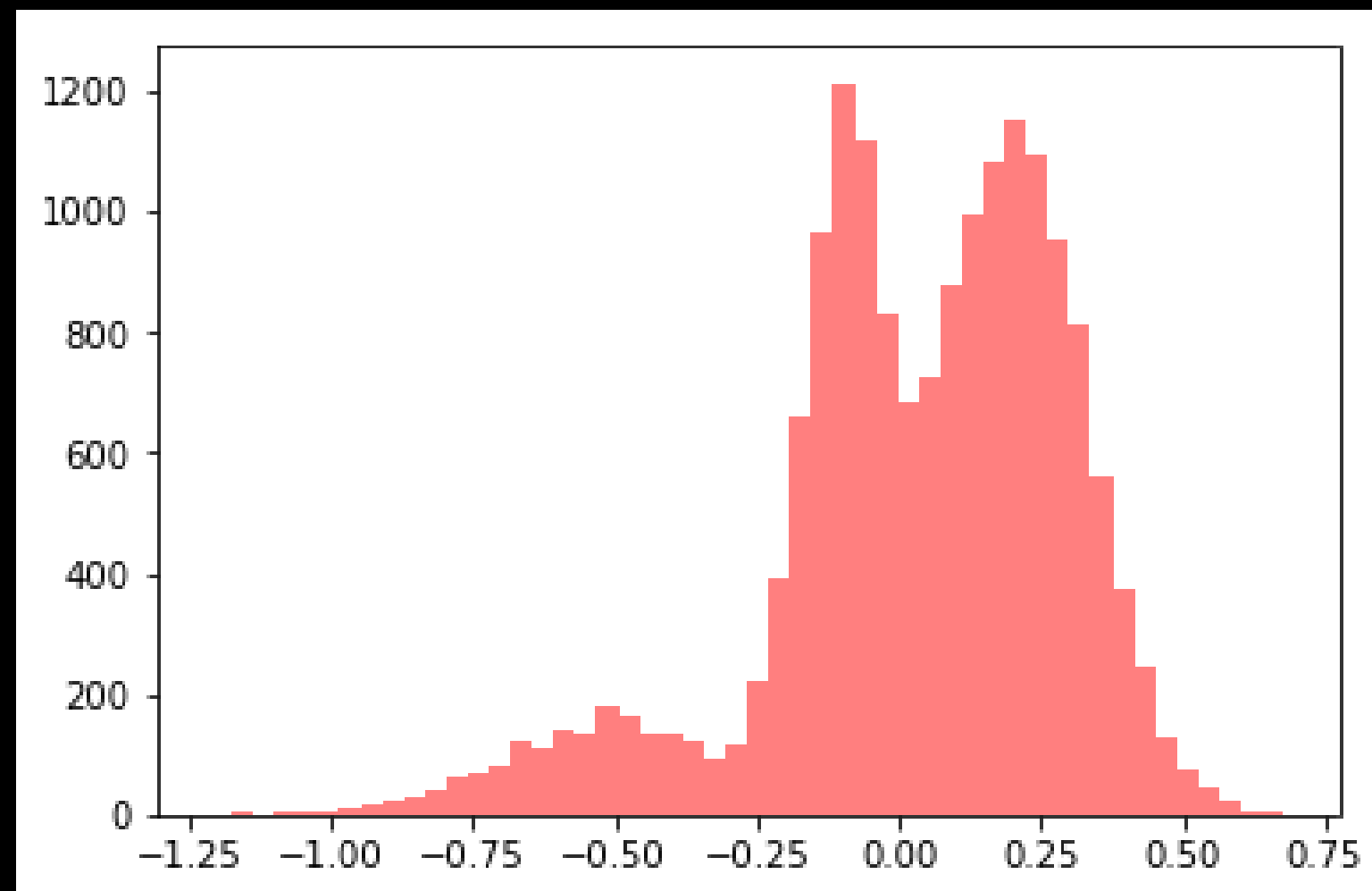


- 구하고자 하는 것

1. 클러스터의 형태 (각 클러스터의 평균과 분산)

2. 데이터가 어떤 클러스터에 속하는지

→ 동시에 구하는 것은 불가능, 두 가지 과정으로 분리해서 수행





1. 각 클러스터마다 **해당 클러스터가 선택될 확률**과 **평균, 그리고 분산**을 랜덤하게 초기화
 - 클러스터가 선택될 확률 = 데이터가 어떤 클래스에 속하는지
 - 평균과 분산 = 클러스터의 형태



2. 변화가 없을 때까지 **모든 데이터에 대해 아래 과정**을 반복

- 클러스터가 선택될 확률, 평균, 분산이 주어짐
- 각 데이터가 어느 클러스터에 들어가는지 계산

→ 클러스터의 형태(평균, 분산)가 주어졌을 때, **데이터가 어떤 클러스터에 들어갈 지** 계산



3. 변화가 없을 때까지 **모든 클러스터에 대해 아래 과정**을 반복

- 각 데이터가 어느 클러스터에 들어가는지가 주어짐
- 각 클러스터마다 선택될 확률, 평균, 분산 계산

→ 데이터가 어떤 클러스터에 들어갈지 주어졌을 때, **클러스터의 형태(평균, 분산)** 계산



- 계산량이 많아 대량의 데이터에 적용하기에 적합하지 않음
- 데이터의 분포가 원형이 아니여도 좋은 성능을 보임
- 데이터의 분포를 가우시안 분포로 가정하기 때문에 이상치에 취약



```
from sklearn.mixture import GaussianMixture
```

```
gmm=GaussianMixture(n_components=3, random_state=100) 모델 생성
```

```
gmm.fit(X) 모델 학습
```

```
gmm.predict(X) 클러스터링 결과 반환
```

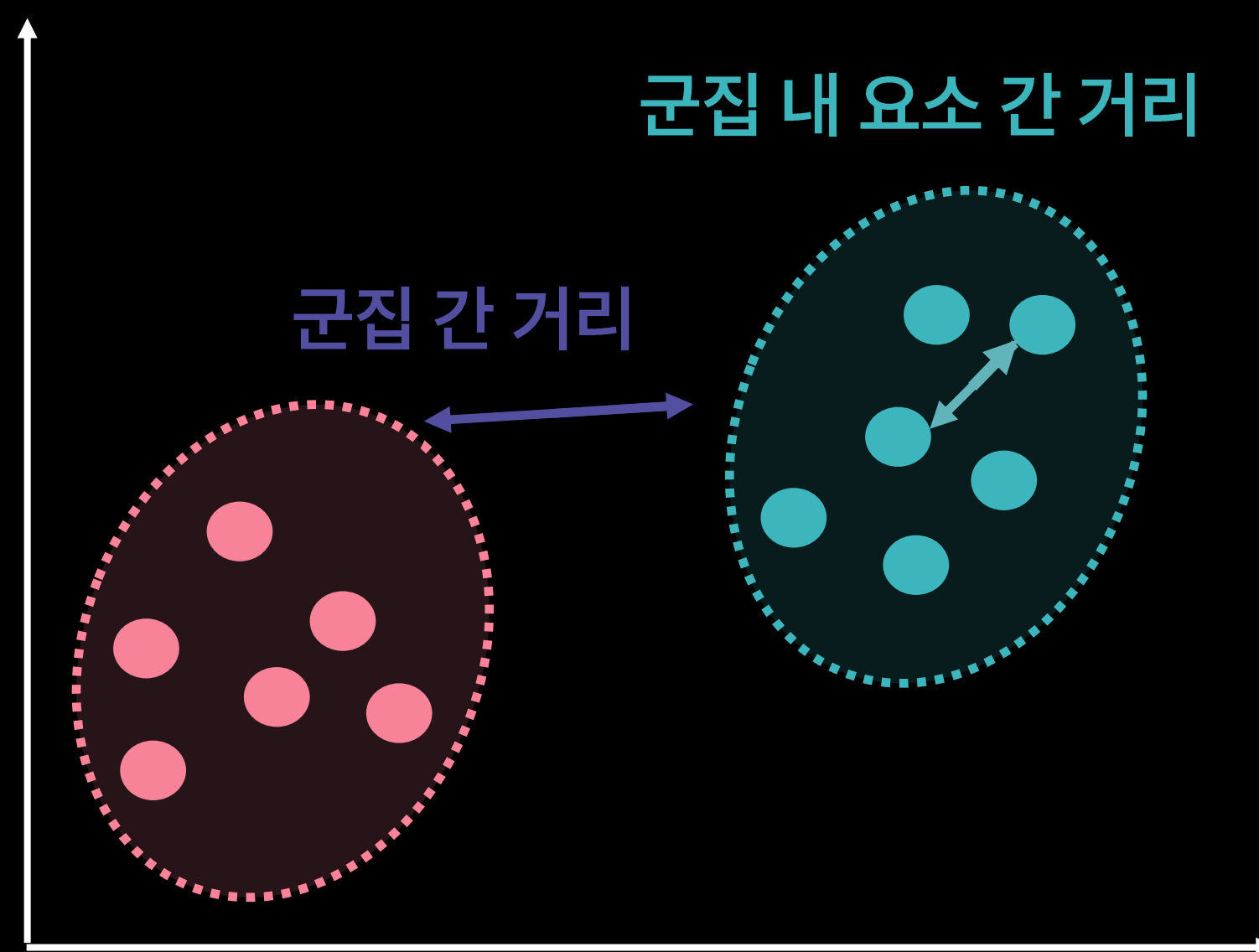
- **n_components = 군집 개수 설정**
- random_state = 분할 과정에서의 무작위성 제어



- **정답이 없기 때문에** 실제값과 예측값의 오차 혹은 단순 정확도 **지표로 평가할 수 없음**
- 군집 간 거리, 군집의 지름, 군집의 분산을 고려하여 **클러스터링 목표 달성 여부 확인**
- 대표적인 평가 지표
 - Dunn Index
 - 실루엣 지표



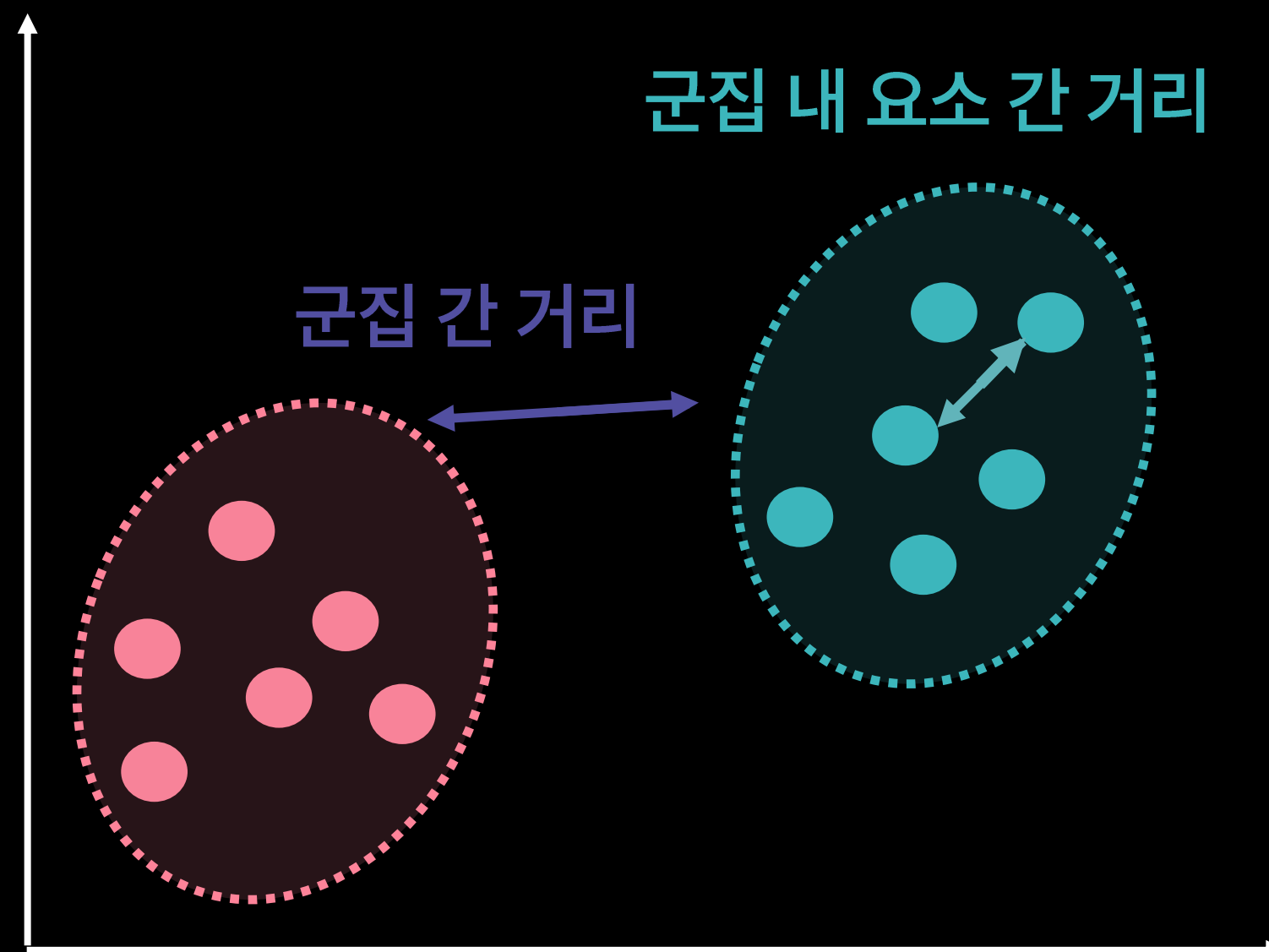
분자값이 클수록 군집 간 거리가 크고,
분모값이 작을수록 군집 내의 데이터들이 모여 있다는 뜻





$$\text{Dunn Index} = \frac{\text{군집 간 거리의 최소값}}{\text{군집 내 요소 간 거리의 최대값}}$$

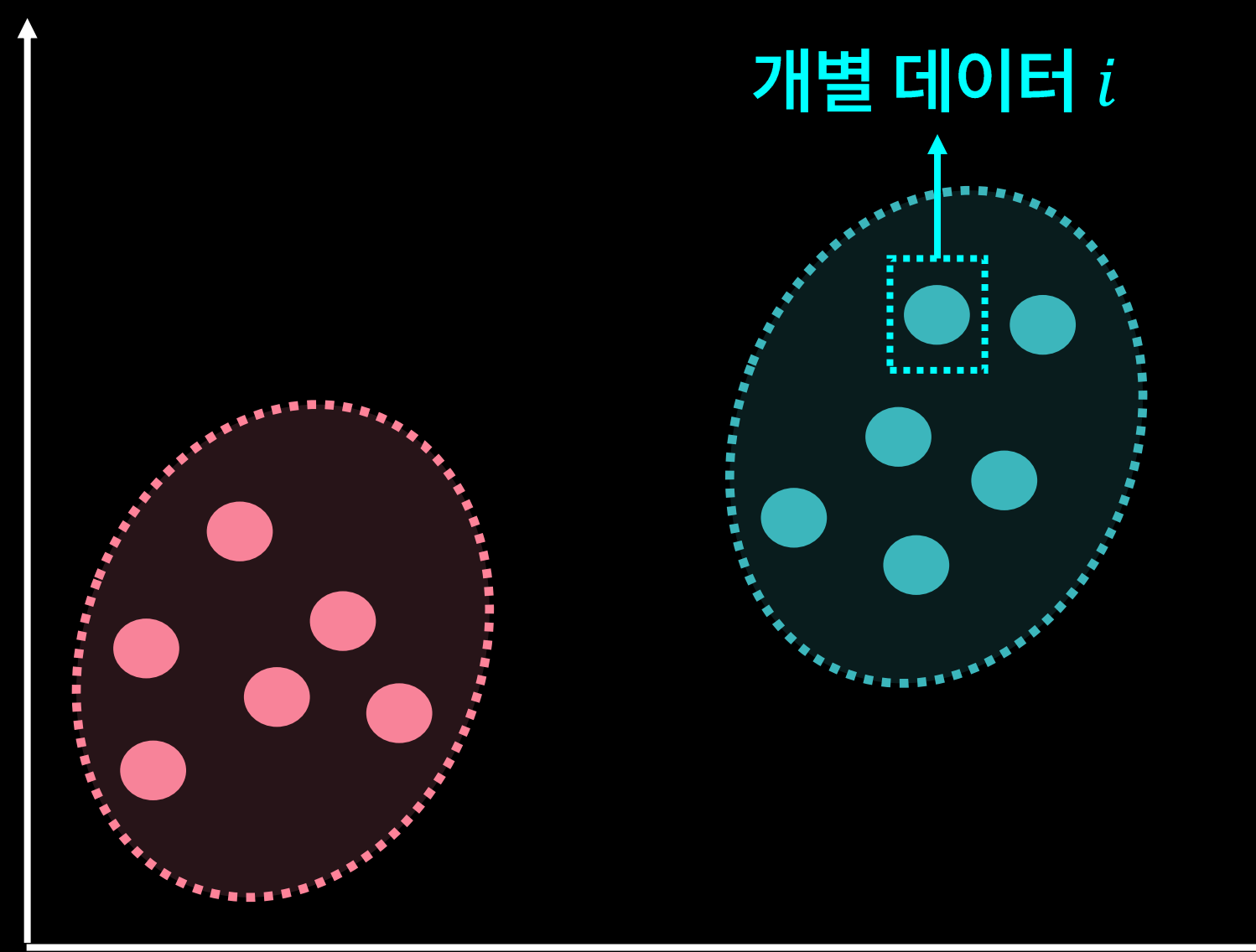
- 클수록 높은 성능을 의미





얼마나 잘 군집화 되었는지에 대한 **정량적 평가**

클러스터의 밀집 정도 계산





$$\text{실루엣 지표} = \frac{a(i) - b(i)}{\max(a(i), b(i))}$$

- $a(i)$: i 번째 데이터가 속한 군집과 가장 가까운 이웃 군집을 택해서 계산
- $b(i)$: i 번째 데이터와 같은 군집에 속한 요소들 간 거리의 평균
- -1에서 1 사이의 값을 가지며, **1에 가까울수록 높은 성능**

4. 차원 축소와 주성분 분석



1 차원

변수1
...
...

2 차원

변수1	변수2
...	...
...	...

3 차원

변수1	변수2	변수3
...	...	...
...	...	...

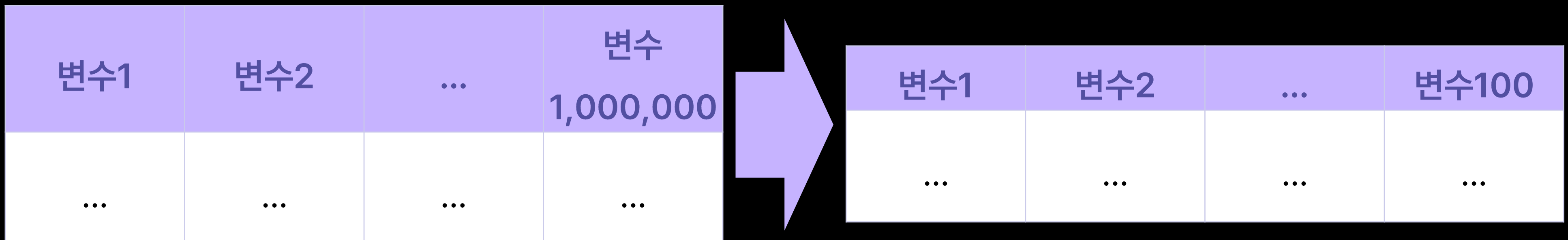
데이터에서 차원은 데이터가 가지는 **특성(또는 변수)**의 수를 의미

- 현재 보유하고 있는 학습 데이터의 변수가 100만개라고 가정
- 만약, 학습 속도 개선과 데이터 압축을 하고자 한다면?

변수1	변수2	...	변수1,000,000
...	...	...	...

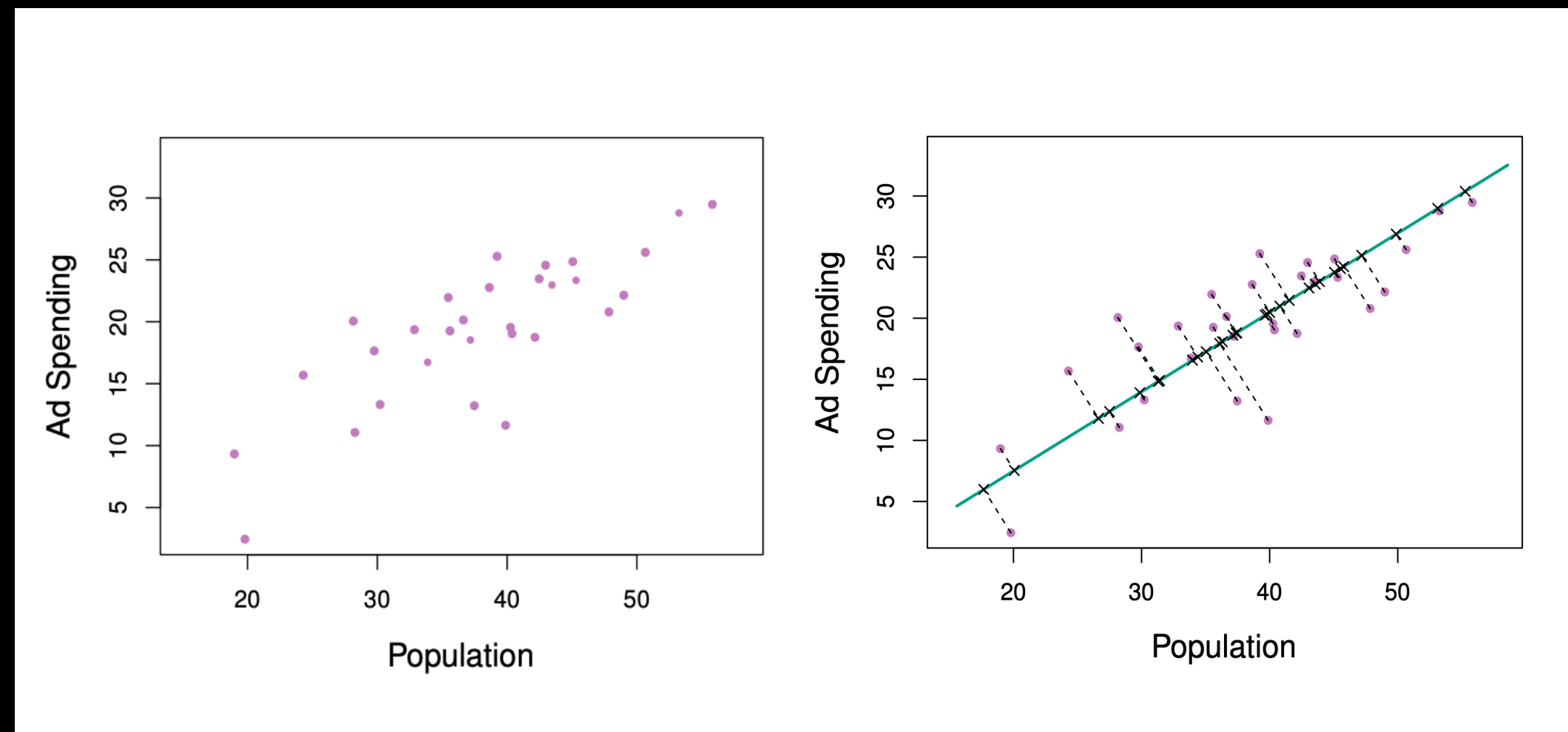


- 문제 정의 : 데이터 변수의 개수를 줄여 데이터의 차원을 줄이면 어떨까?
- 데이터 : 100만 개의 변수를 가진 데이터
- 목표 : 학습 속도 개선 및 데이터 압축하기
- 해결 방안 : **차원 축소(Dimensionality Reduction) 알고리즘**



고차원의 데이터를 저차원으로 줄이는 알고리즘

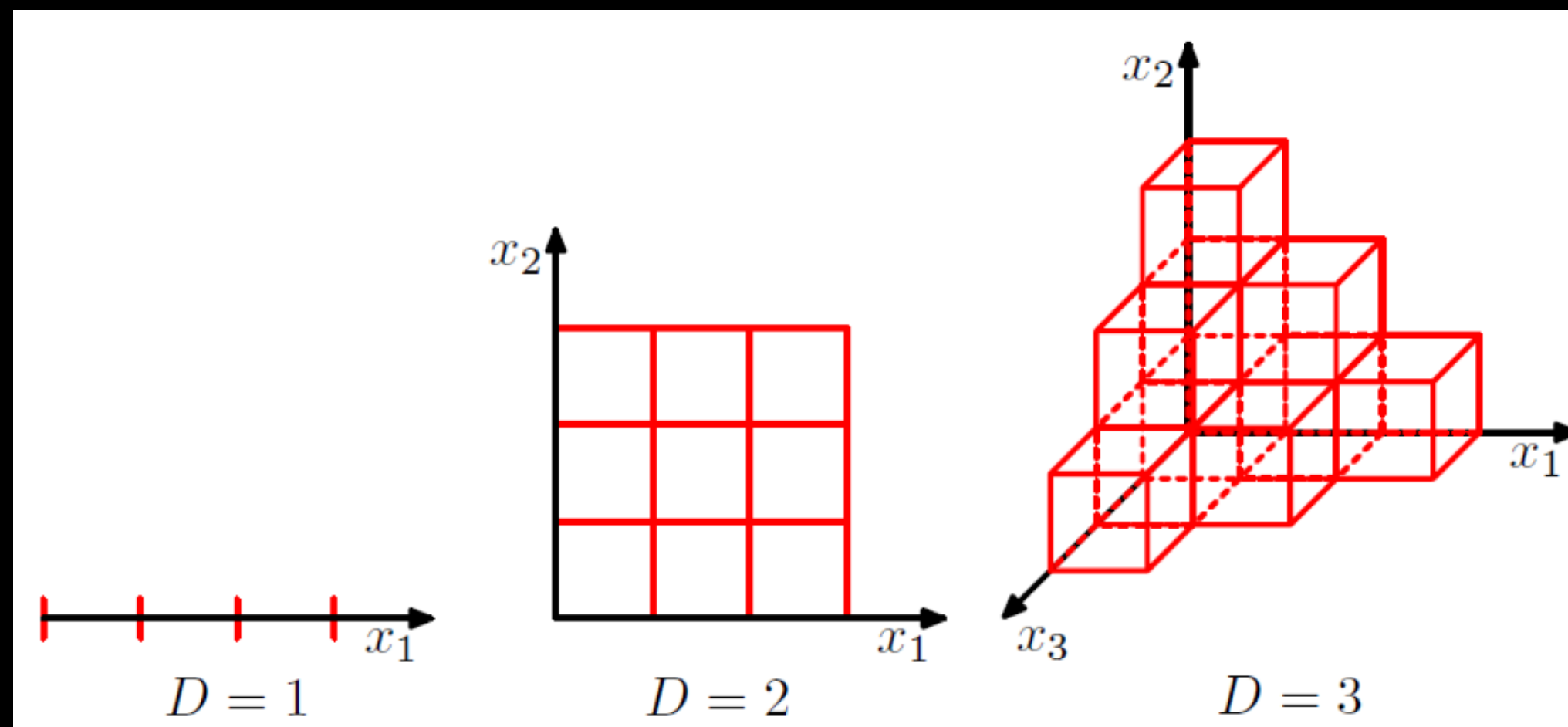
- 엄청나게 많은 변수를 가지고 있는 고차원의 데이터에서는 차원의 저주가 발생할 가능성이 높음





차원이 높을수록 학습에 요구되는 데이터의 개수도 증가

- 고차원일 때, 적은 개수의 데이터로만 차원을 표현하는 경우 **과적합(Overfitting)** 발생 가능



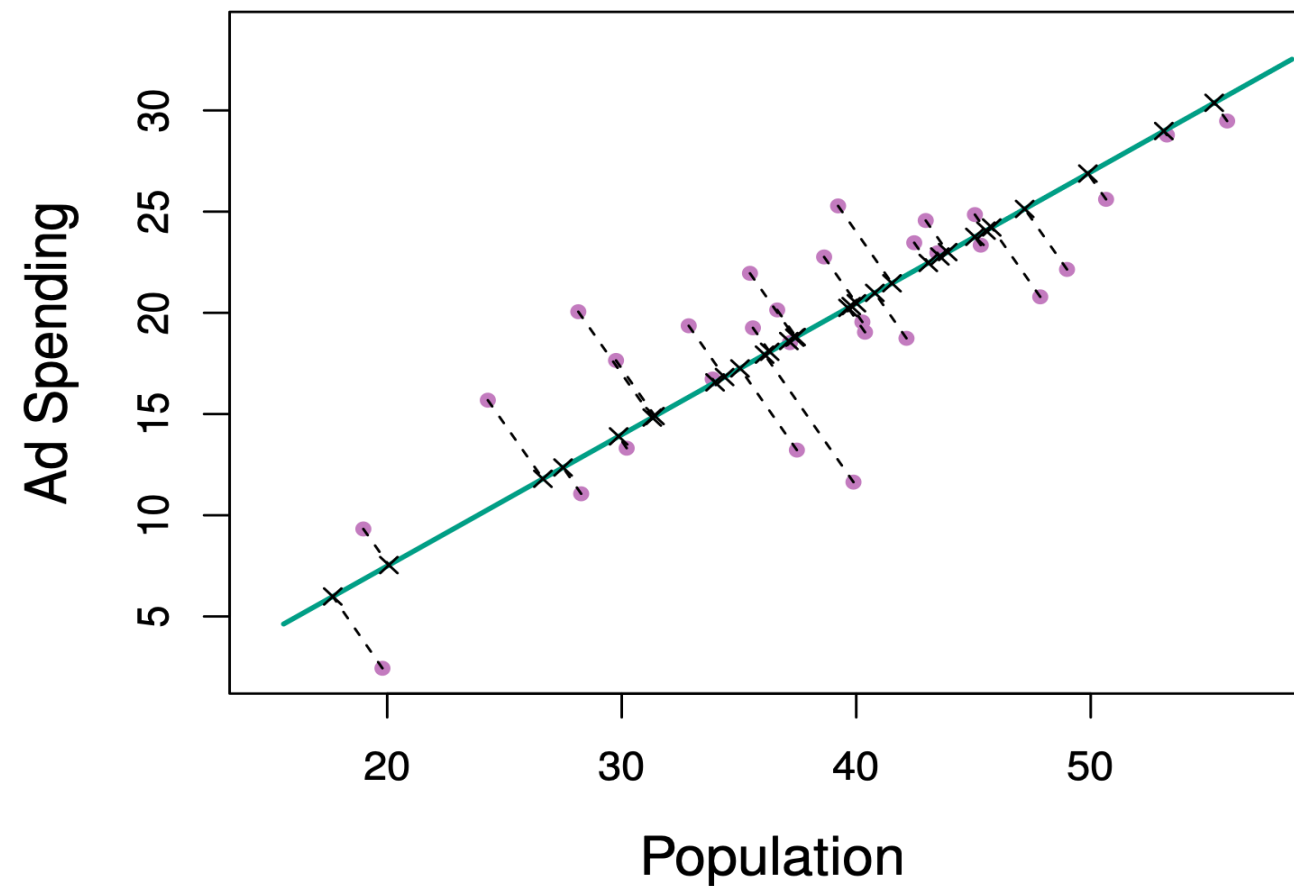


- 차원의 저주 발생 방지
- 모델 학습 속도 및 성능 향상
- 대표적인 차원 축소 알고리즘
 - 주성분 분석(Principle Component Analysis)
 - t-SNE(t-Stochastic Neighborhood Embedding)



고차원의 데이터를 가장 잘 설명할 수 있는 주성분을 찾는 방법

- 차원을 축소하면서도 원본 데이터의 특징을 가지고 있을 수 있도록 함



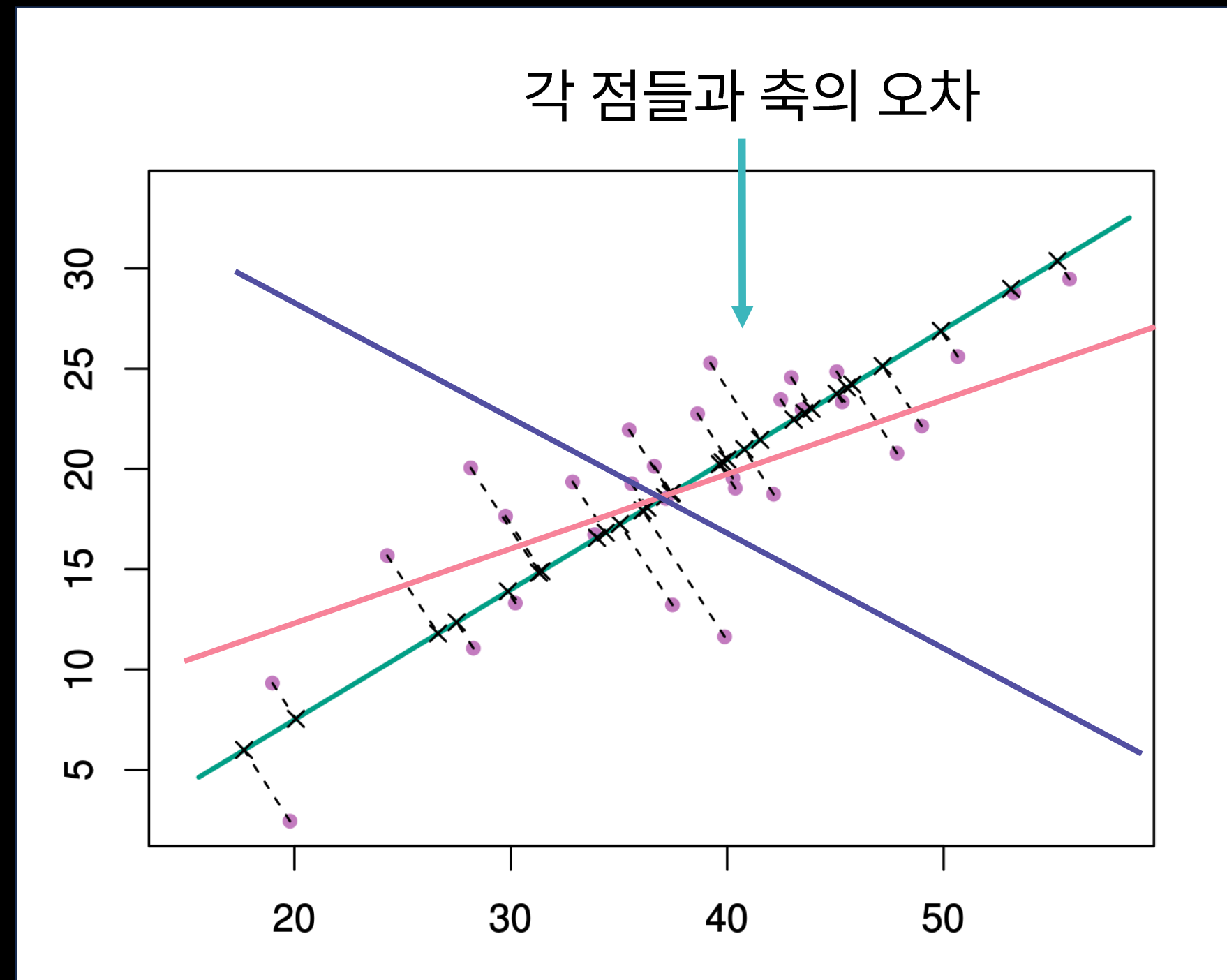


- 원본 데이터의 특징을 가지고 있다.
 - = 원본 데이터와의 **차이를 최소화**해야 함
 - = 원본 데이터와의 **차이를 최소화 하는 축(주성분)**을 찾아야 함



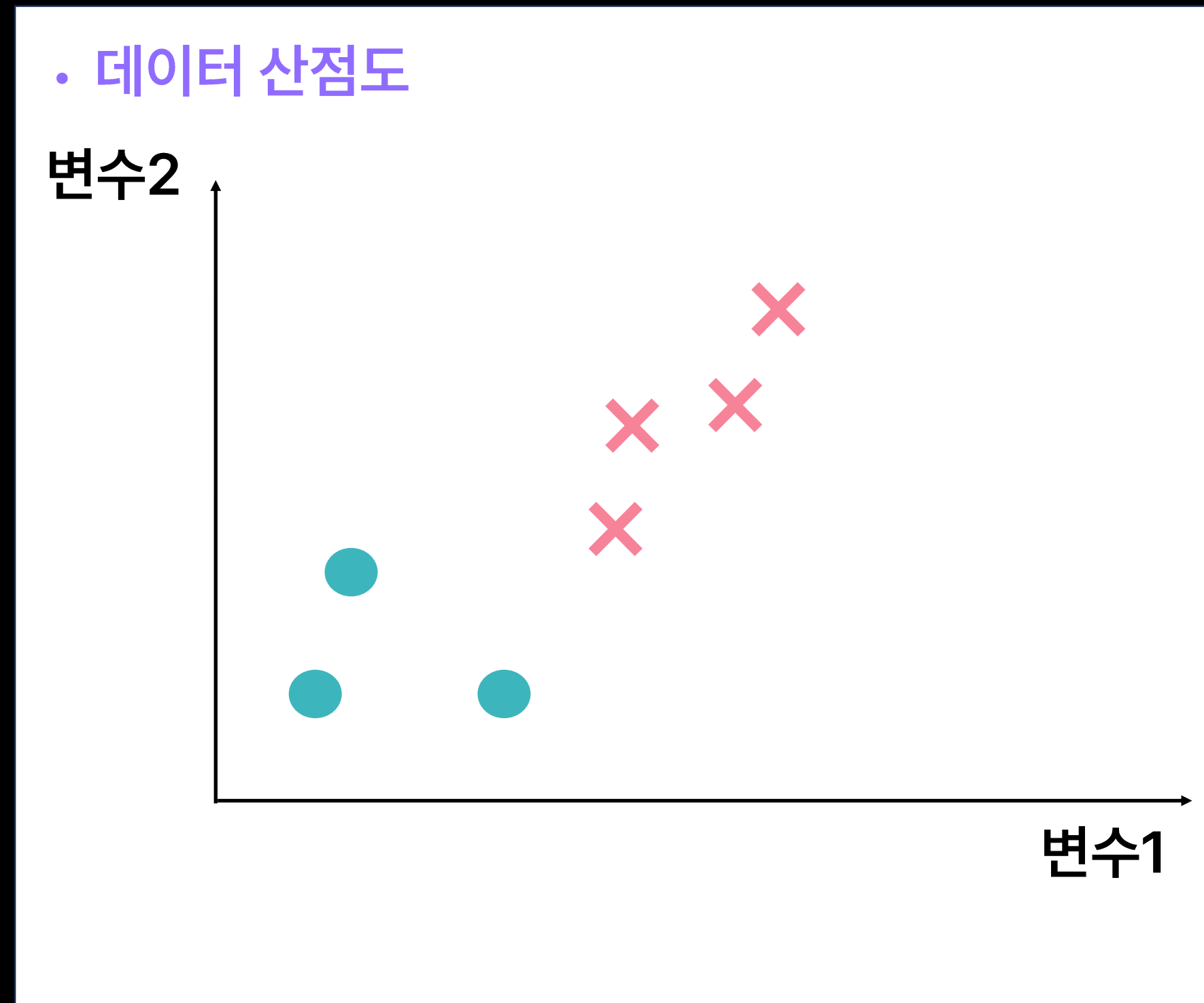
- 2차원 데이터를 1차원으로 차원 축소할 경우

→ 여러 갈래의 축을 확인해보며 **각 점들과 축의 오차가 가장 작은 축을 중심으로** 데이터를 모음





- 2차원 데이터를 1차원으로 차원 축소하는 경우
- 예시 데이터 분포가 아래와 같음





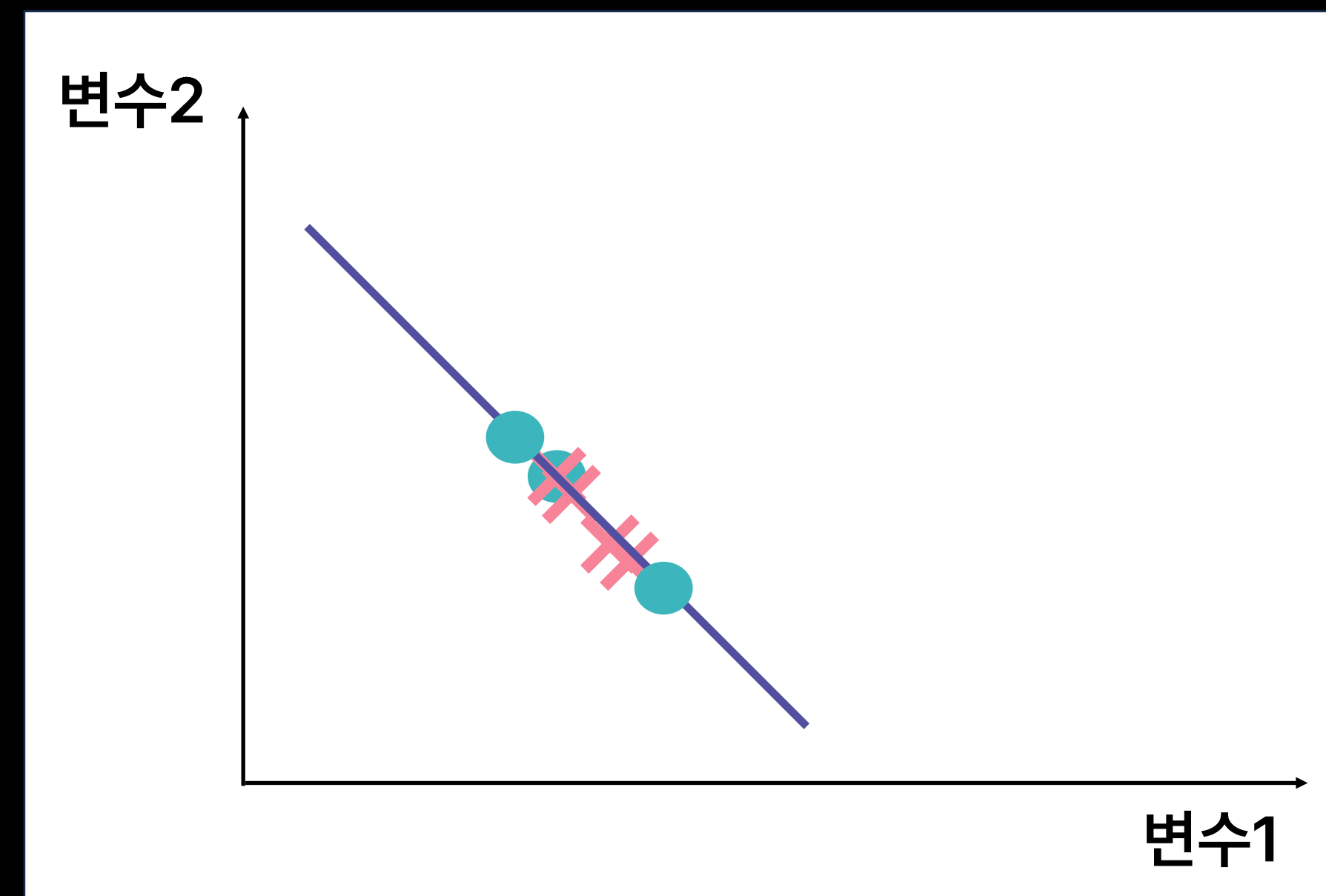
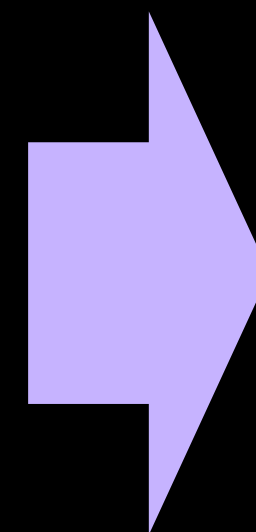
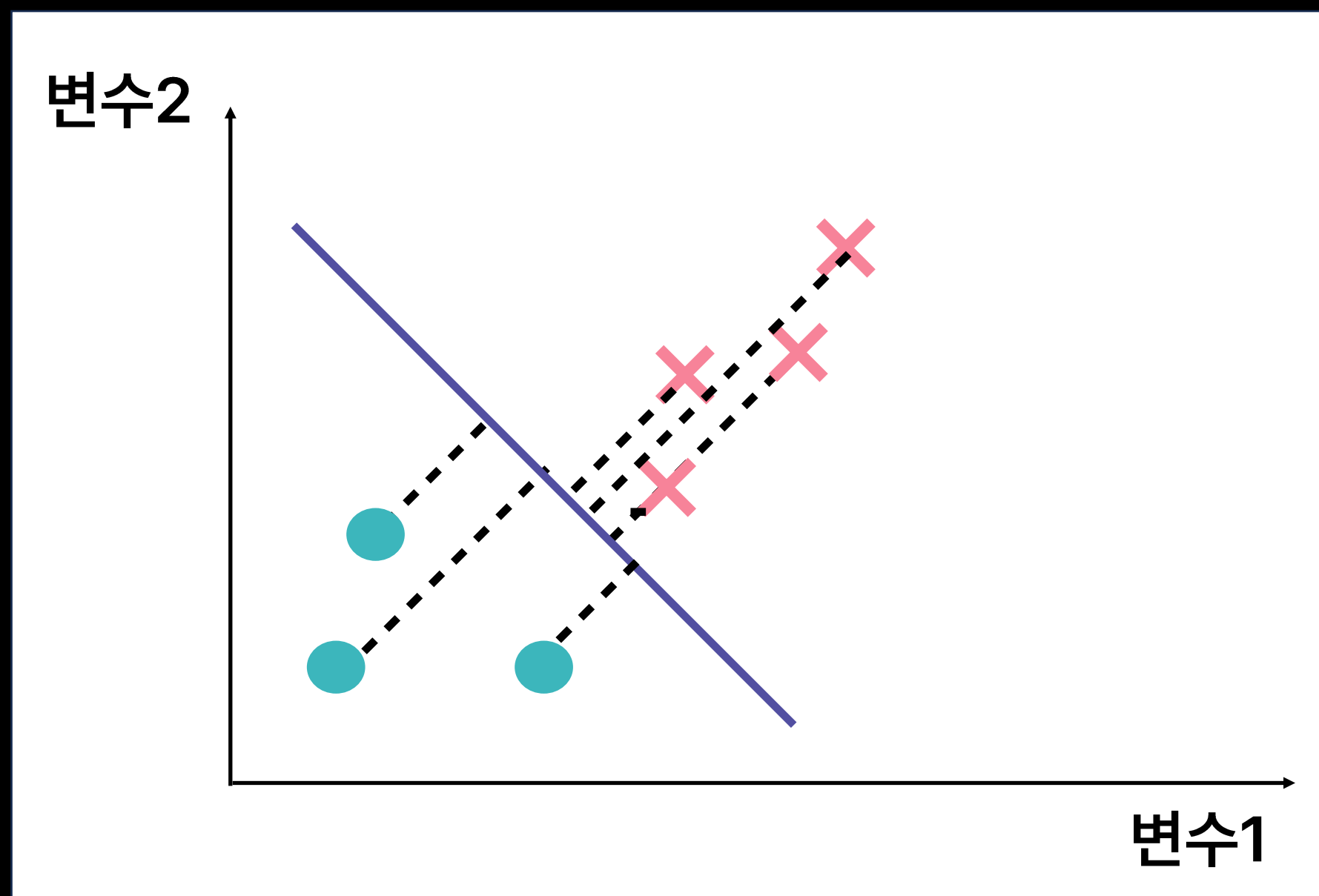
주성분 분석(PCA) 원리 예시

머신러닝 II

03 비지도학습

4. 차원 축소와 주성분 분석

- Case 1 : 각 점들과 축의 오차가 **큰** 축





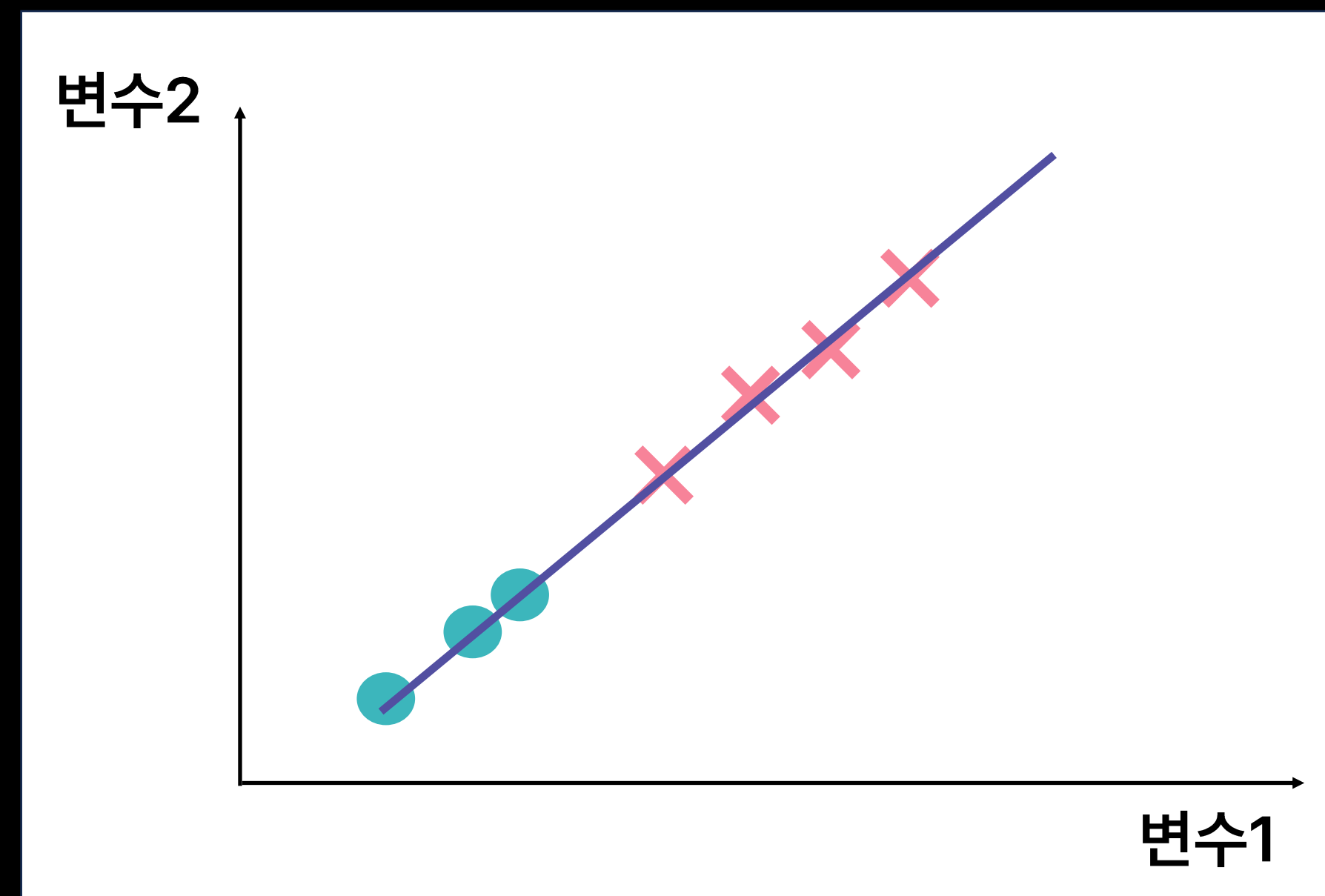
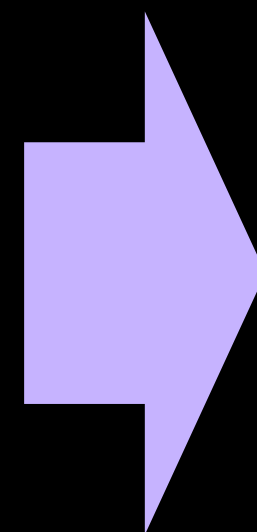
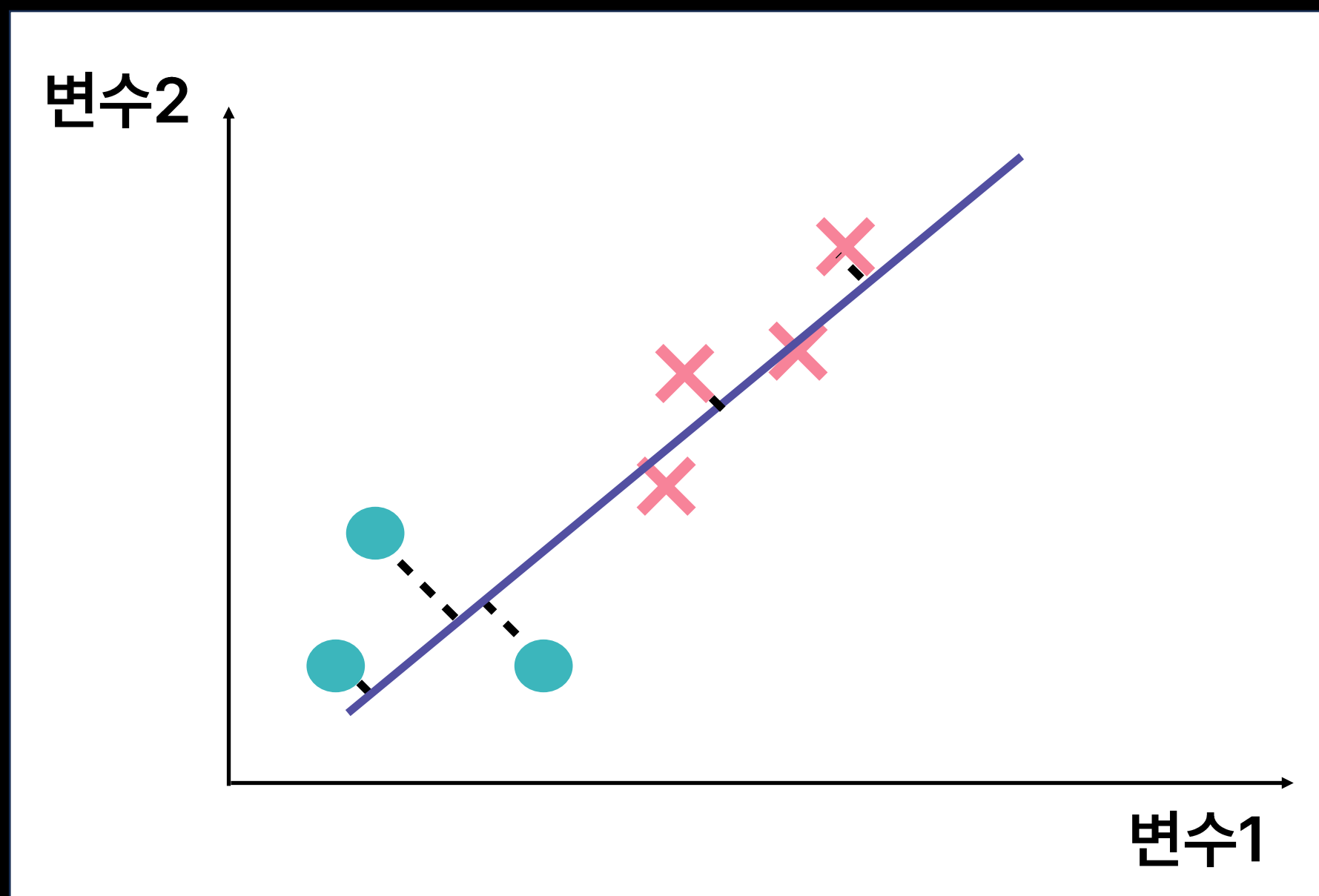
주성분 분석(PCA) 원리 예시

머신러닝 II

03 비지도학습

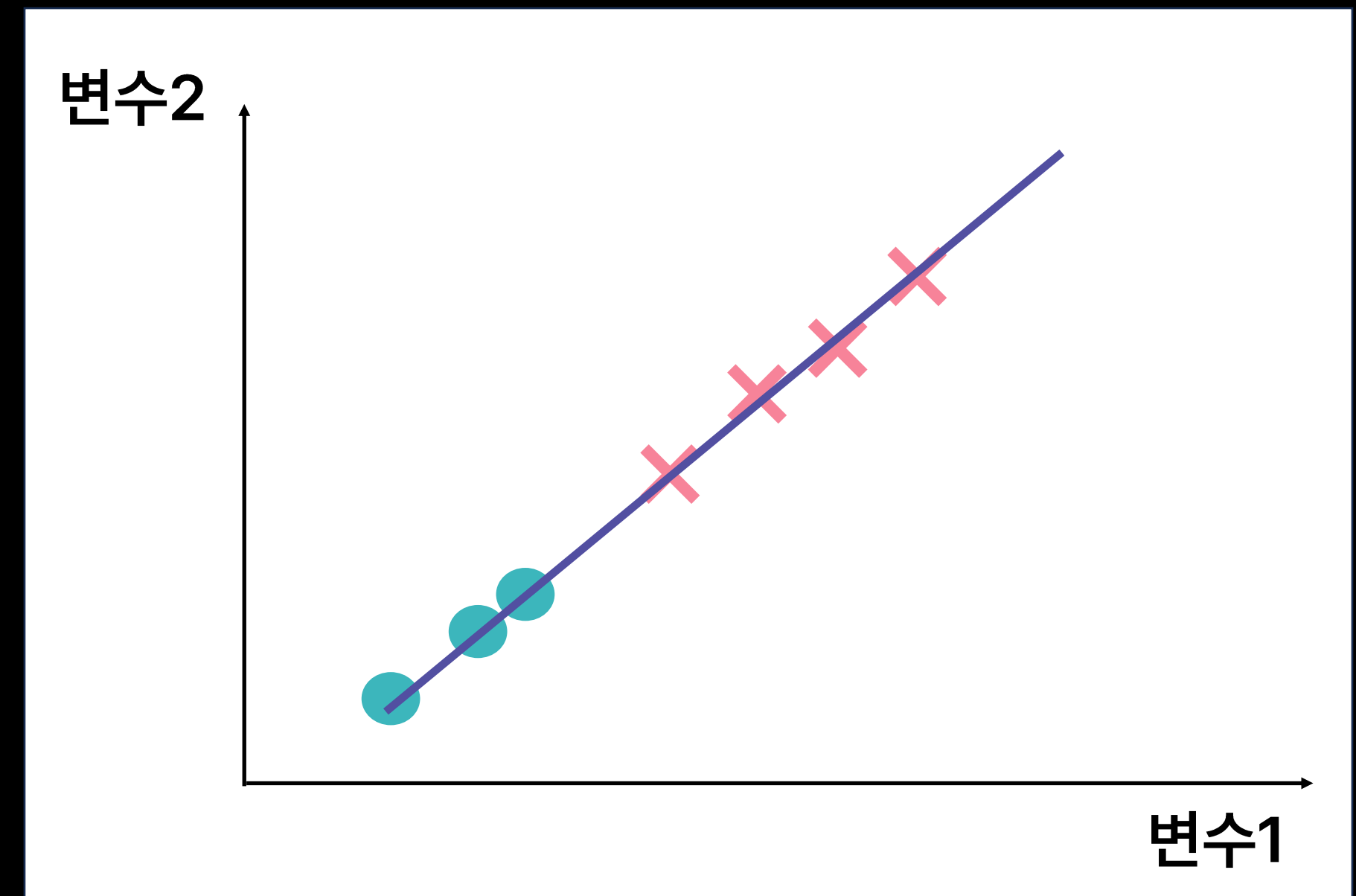
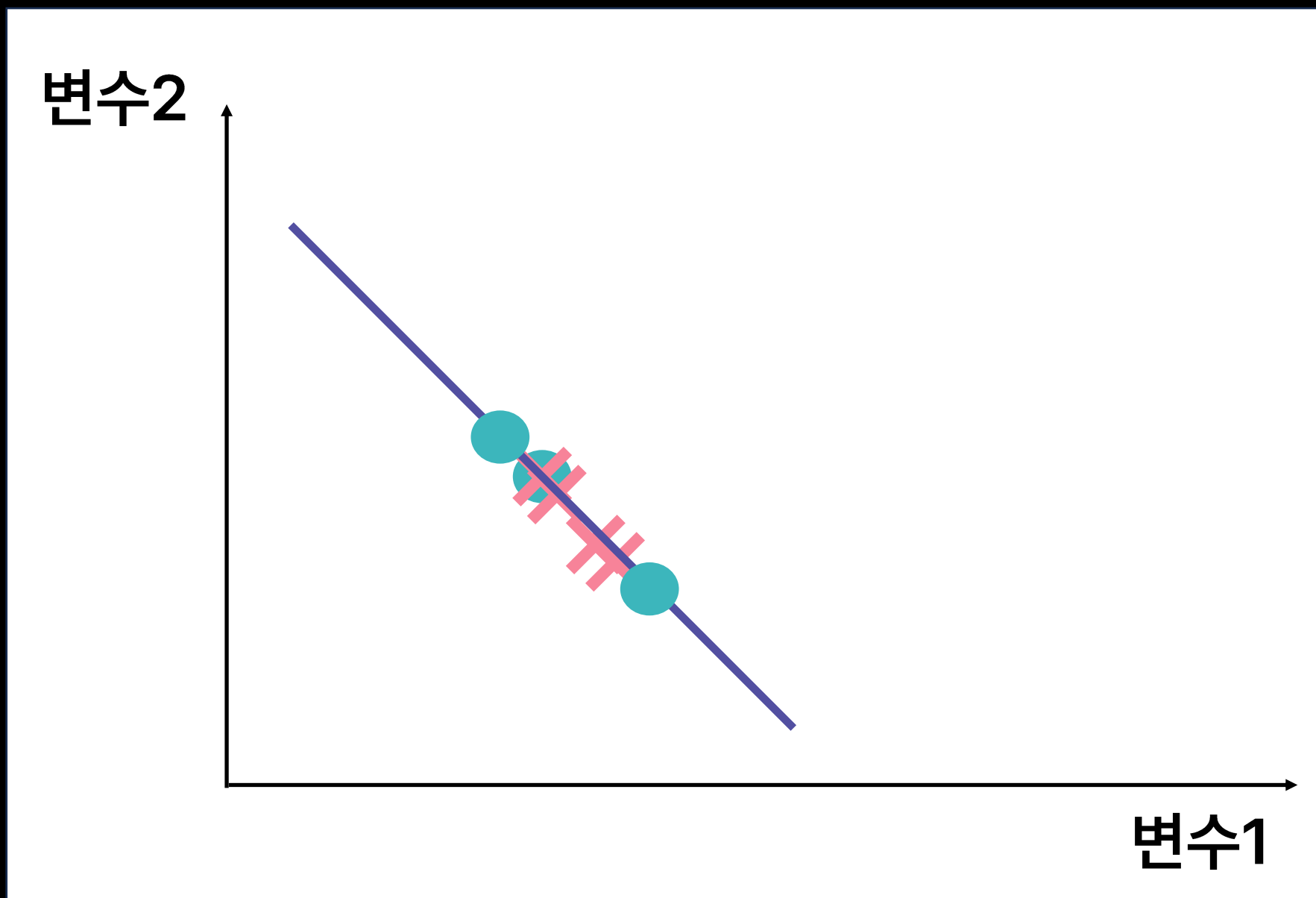
4. 차원 축소와 주성분 분석

- Case 2 : 각 점들과 축의 오차가 **작은** 축





- 모델 학습 전 **변수들의 단위를 스케일링** 해주는 것이 좋음
- 고차원의 데이터를 함축적으로 표현하기 때문에 **직관적인 해석이 어려울 수 있음**
- 대용량 **고차원 데이터 압축 시 유용**하게 활용됨





```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2) 모델 생성
```

```
pca.fit(X) 모델 학습
```

```
pca.transform(X) X를 차원 축소한 결과 데이터 반환
```

- **n_components = 군집 개수 설정**

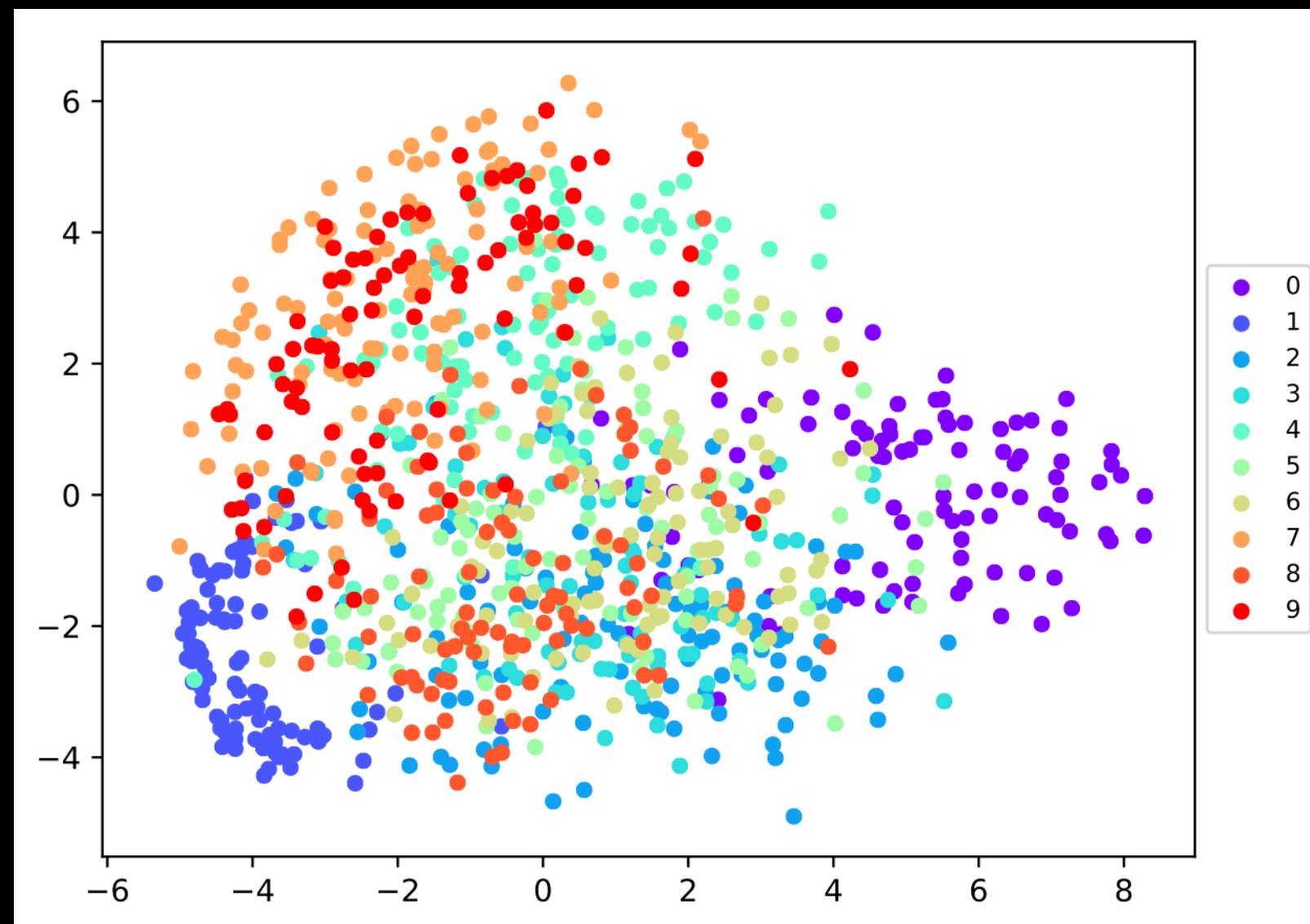
5. t-SNE



- 문제 정의 : 고차원 공간에서 데이터들의 군집을 구별하여 시각화하고 싶다면?

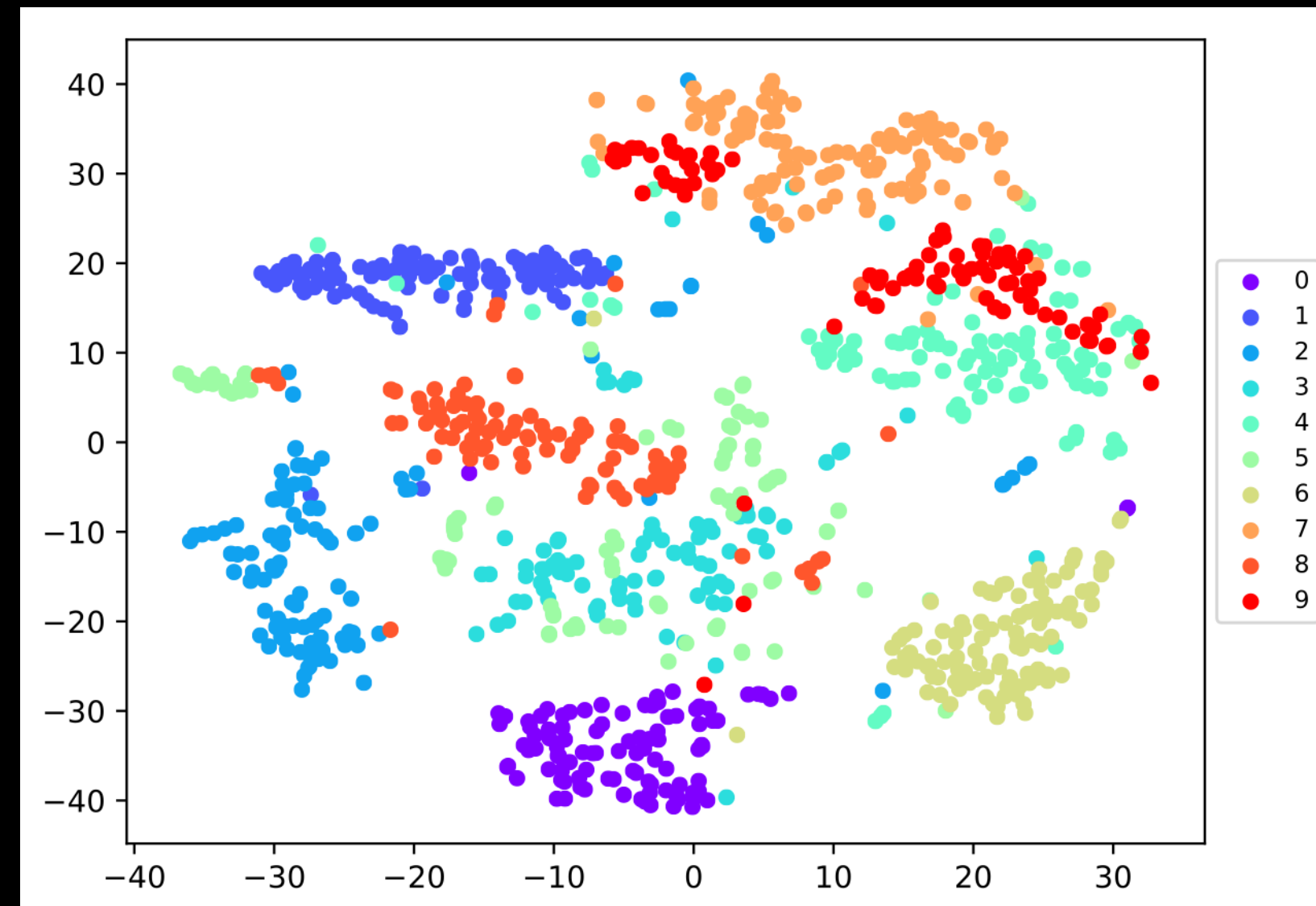
PCA 사용시, 함축적 표현으로 시각화를 진행하기 어려움

- 해결 방안 : **t-SNE(t-Stochastic Neighborhood Embedding)** 알고리즘 활용



고차원의 공간에 존재하는 데이터 간의 거리를 최대한 유지하며 차원을 축소하는 방법

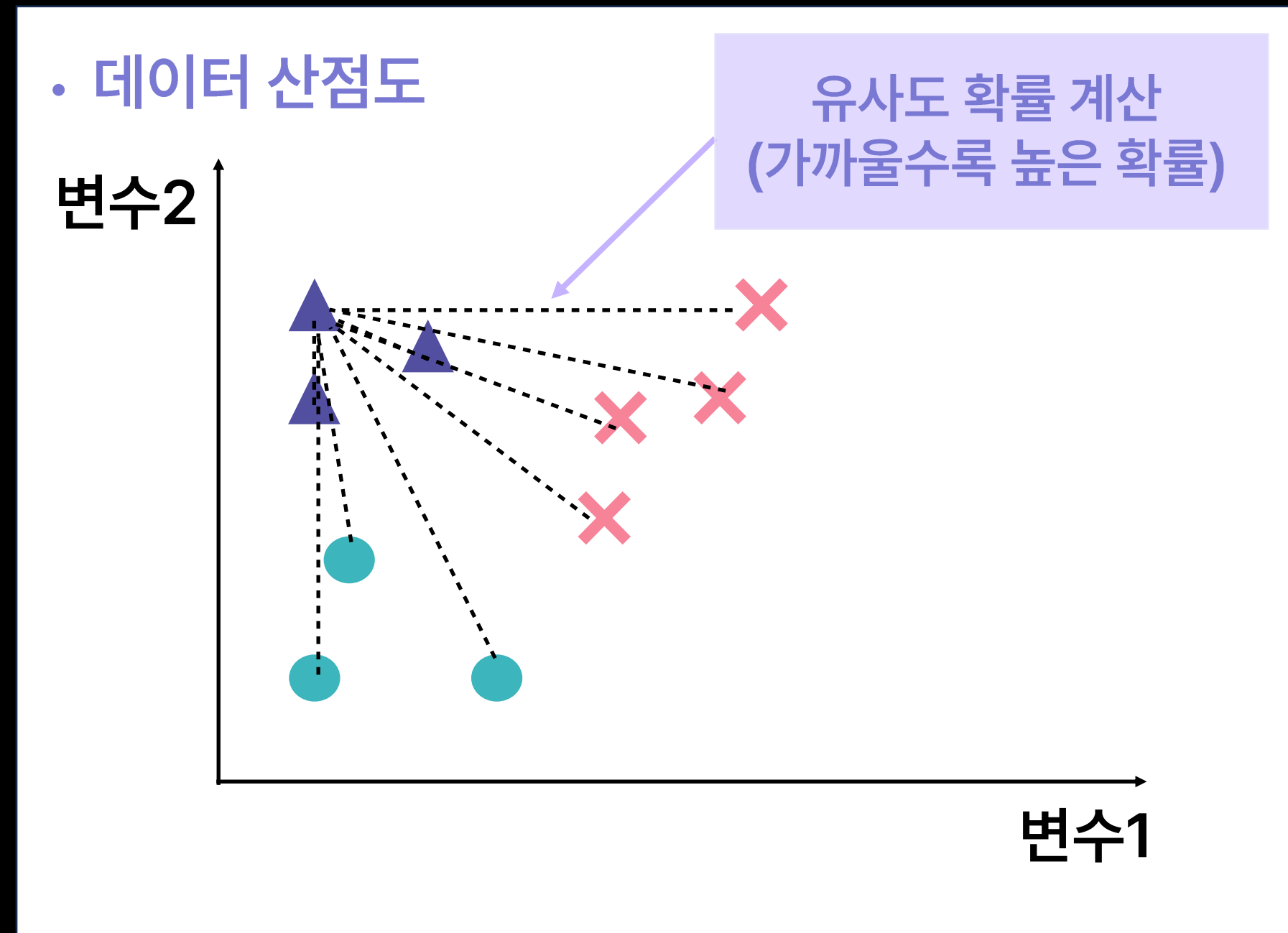
- 기존 데이터 공간에서 서로 가까이 있는 데이터는 차원이 줄어든 공간에서도 가까이 있어야 함





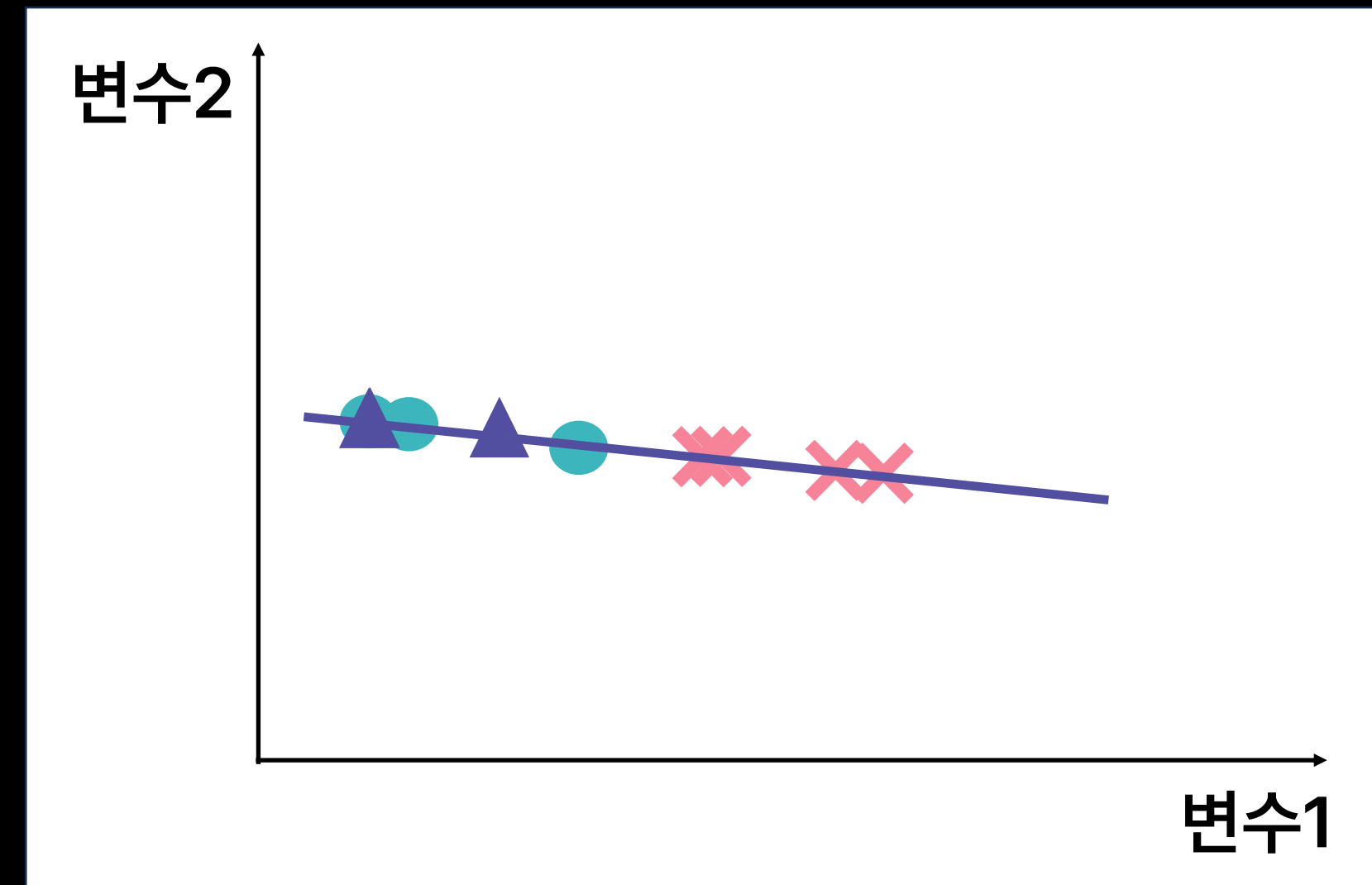
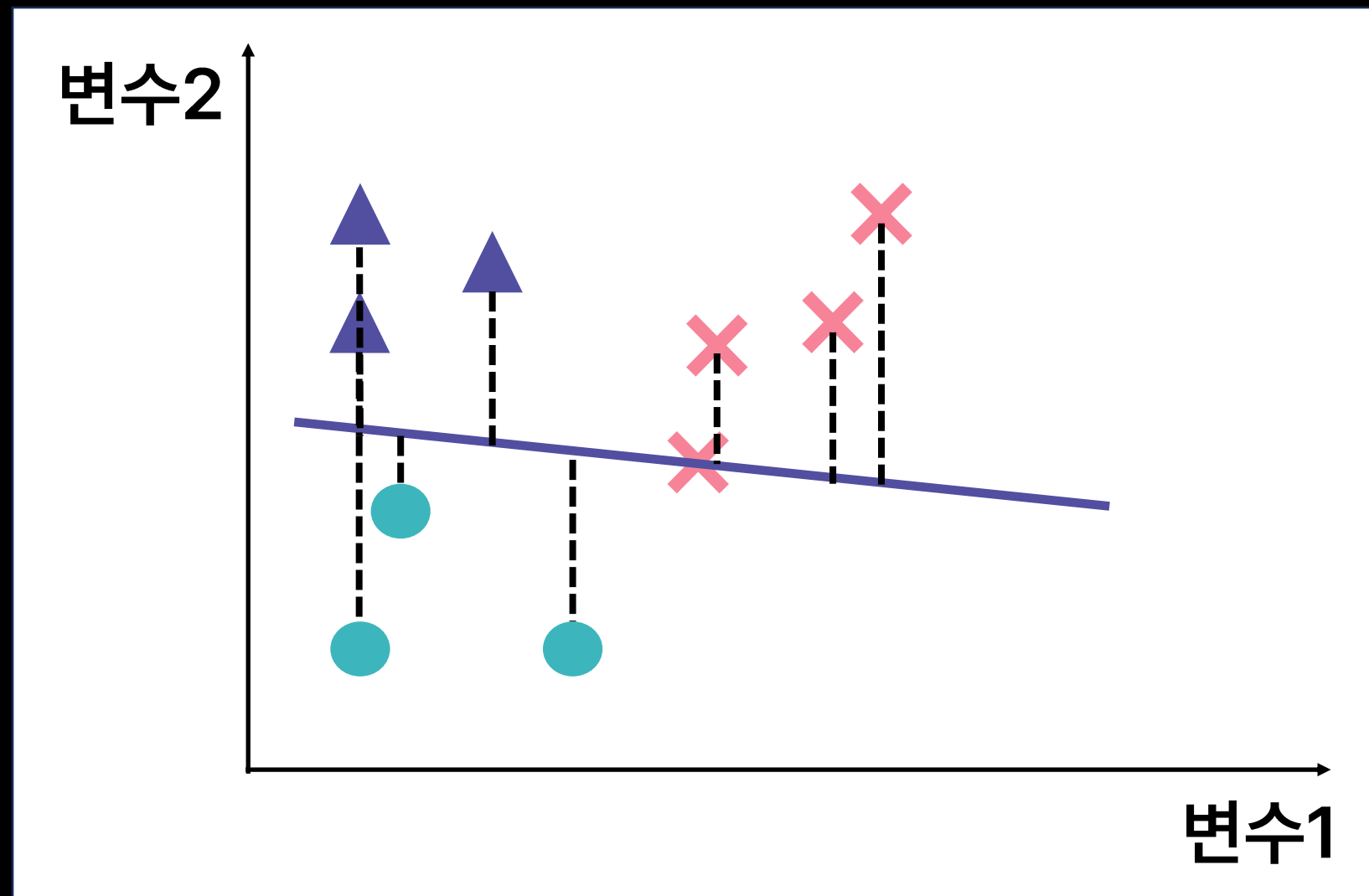
1. 각 데이터마다 자기 이외의 데이터와의 유사도 확률 계산

→ 여러 갈래의 축을 확인해보며 **각 점들과 축의 오차가 가장 작은 축을 중심으로** 데이터를 모음



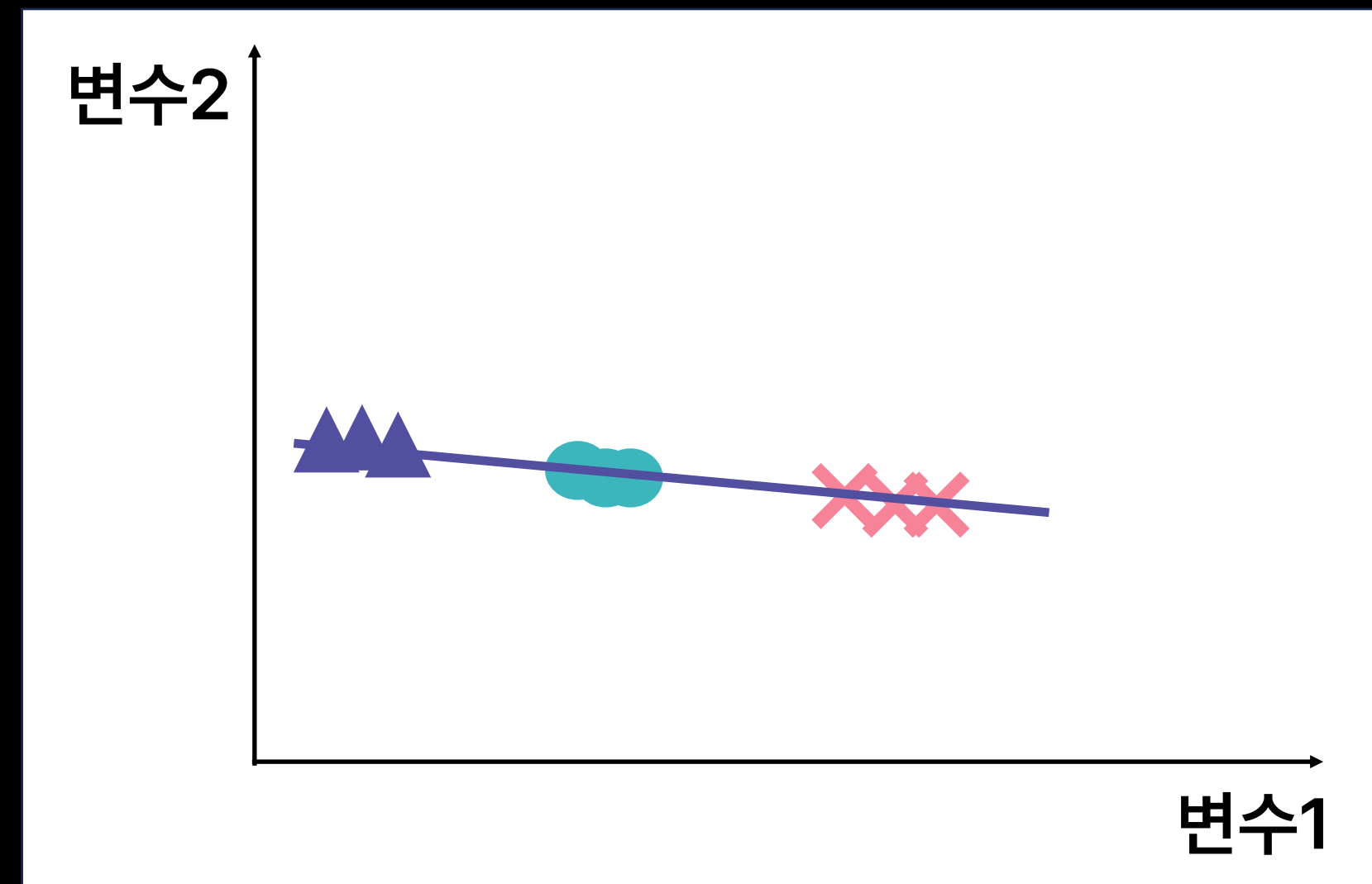
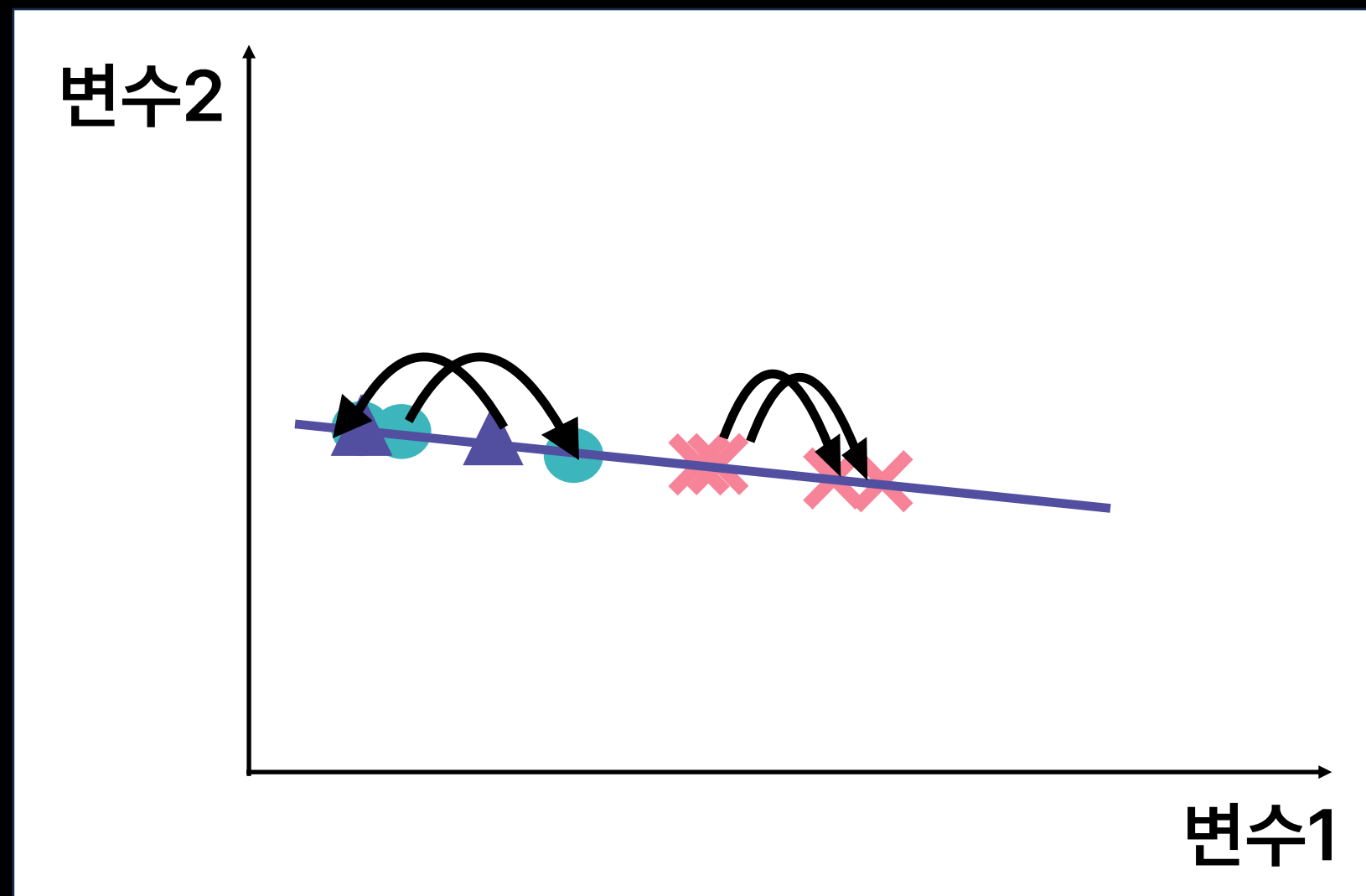


2. 저차원으로 축소



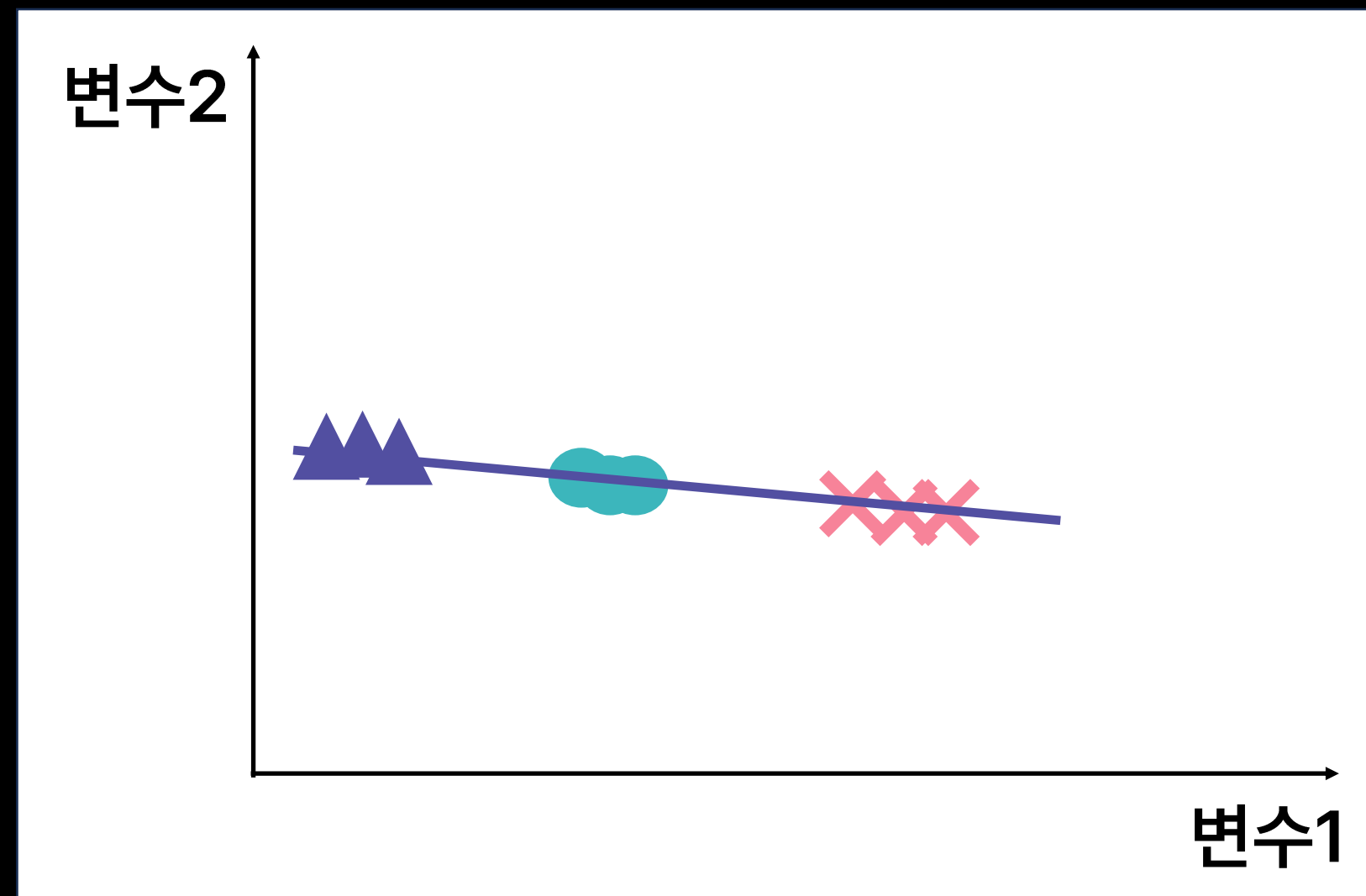


2. 초기에 계산했던 유사도 확률분포를 바탕으로 각 데이터를 이동





- 데이터 간 거리 유지를 통해 차원 축소 이후에도 객체 간 구별이 가능
- 계산시 마다 값이 지속적으로 변경되어 예측을 위한 학습 데이터로는 사용 불가
- 고차원 데이터의 시각화를 위해 활용됨





```
from sklearn.manifold import TSNE  
  
tsne = TSNE(n_components=2) 모델 생성  
  
tsne.fit_transform(X)
```

- **n_components = 군집 개수 설정**
- fit_transform()
 - fit() 과 transform() 을 함께 진행하는 함수
 - PCA에도 사용 가능